Cardiff Metropolitan University
B.Sc. (Hons) in Data Science
Assignment Cover Sheet

Student Details (Student should fill the content)				
Name	Mohamed Hamaad Fahim			
Student ID	st20289069			
Scheduled unit details				
Unit code	CIS6027			
Unit title	Data Visualization Grammar and Idioms			
Unit enrolment details	Year	3		
	Study period			
Lecturer				
Mode of delivery	Full Time			
Assignment Details				
Nature of the Assessment	Individual coursework: Written report + interactive dashboards			
Topic of the Case Study	Seattle Airbnb open data: exploratory and explanatory Dash dashboards			
Learning Outcomes covered	LO1–LO4			
Word count	3929			
Due date / Time	09-02-2026			
Extension granted?	Yes	No	Extension Date	
Is this a resubmission?	Yes	No	Resubmission Date	
Declaration				
I certify that the attached material is my original work. No other person's work or ideas have been used without acknowledgement. Except where I have clearly stated that I have used some of this material elsewhere, I have not presented it for examination / assessment in any other course or unit at this or any other institution				
Name/Signature		Date		
Submission				
Return to:				
Result				
Marks by 1st Assessor		Name & Signature of the 1st Assessor		**Agreed Mark**
Marks by2nd Assessor		Name & Signature of the 2nd Assessor		
Comments on the Agreed mark				

Table of Contents

Cardiff Metropolitan University 1
1. Introduction 1
2. Task 1: Advanced Grammar and Idioms of Visualization 1
2.1. Grammar vs idioms: definitions and impact on encoding, perception, storytelling 1
2.2. Worked comparison: temporal data, the same calendar rows, two encodings 2
2.3. Worked comparison: spatial data, coordinate grammar vs the map idiom 4
2.4. Trade-offs: expressive power vs cognitive accessibility 6
2.5. Business bias: how businesses unintentionally mislead via grammar or idioms 7
3. Task 2: Multi-grain Data, Abstraction, and Semantic Layering 7
3.1. Domain and three data types 7
3.2. Grain journey: atomic to aggregate to insight 8
3.3. Semantic layering and the idiom that fits each level 9
Table 1. Semantic layering and the idiom that fits each level 9
3.4. Critical review: where grain misalignment breaks the chain 10
4. Task 3: Designing Idiomatic, Multi-perspective Dashboards 11
4.1. The data and its three required components 11
4.2. Dashboard 1: exploratory (app/exploratory.py, port 8050) 11
4.3. Dashboard 2: explanatory (app/explanatory.py, port 8051) 12
4.4. Critical evaluation: grammar choices, idiom choices, and the decisions they shape 13
5. Task 4: Future of Visualization Idioms with Emerging Tech 15
5.1. Two emerging technologies 15
5.2. How they extend, disrupt, or complement grammar and idioms 15
5.3. Case study: a conversational copilot on this dataset 15
5.4. Ethical and cognitive risks 16
5.5. Forward-looking: the next 5–10 years 16
6. References 17
7. Appendix A: Data Lineage & Known Limitations 18
8. Appendix B: Source Code 20

List of Tables

Table 1. Semantic layering and the idiom that fits each level..... 9

List of Figures

Figure 1. Idiom-first: blocked rate by calendar date..... 3
Figure 2. Grammar-first: blocked rate by booking horizon..... 3
Figure 3. Idiom-first: listings over a basemap..... 5
Figure 4. Grammar-first: listings as bare coordinates..... 6
Figure 5. Row collapse, L0 to L2 (log scale)..... 8
Figure 6. Treemap drill-down by neighbourhood..... 11
Figure 7. What-if scenario, Belltown..... 12
Figure 8. Sankey: group to room type to price tier..... 13

1. Introduction

This report uses the Seattle Airbnb Open Data dataset (Kaggle: airbnb/seattle) as the common domain for all four tasks. The dataset contains 3,818 listings, 1.39 million calendar availability records, and 84,800 free-text reviews, covering host, listing, calendar, and review tables with varying structures and levels of detail. It meets Task 2's requirement for structured, semi-structured, and unstructured data without any simulated data. The listings.csv file provides structured tabular data while also containing semi-structured fields, such as amenities and host_verifications stored as nested lists within cells. The reviews.csv file provides unstructured data in the form of free-text reviews.

A reusable semantic layer was created once in src/build_semantic_layer.py and then used across all subsequent tasks. The raw CSV files are cast, cleaned, and transformed into atomic (L0), aggregate (L1), and KPI (L2) Parquet tables, with the data lineage documented in data/lineage.json. For Task 3, two Dash dashboards (app/exploratory.py and app/explanatory.py) were developed using this semantic layer. Some of their visualisations are also reused as examples in Tasks 1 and 2 instead of being recreated, as the same underlying data is represented in different ways to support the discussion of visualisation grammar and idioms.

One data-quality issue appears across multiple tasks and is therefore documented here once: calendar.csv does not contain a full year of repeated observations. Instead, it represents a single forward-looking snapshot collected on 4 January 2016. This limitation directly affects which metrics can be reliably calculated (see Appendix A) and serves as the main case study for Task 1.

2. Task 1: Advanced Grammar and Idioms of Visualization

2.1. Grammar vs idioms: definitions and impact on encoding, perception, storytelling

A grammar of graphics is a compositional approach to visualisation. Data passes through a defined sequence involving statistical transformations, geometric marks, visual encodings such as position, colour, or size, scales, and guides. A chart is therefore created by combining independent components rather than selecting a predefined chart type (Wilkinson, 2005; Wickham, 2010). Vega-Lite and ggplot2 follow this principle by defining charts through encoding channels linked to data fields. The analyst selects the individual components, and the final visualisation emerges from their combination.

An idiom takes a different approach by treating a visualisation as a familiar, pre-assembled pattern, such as a timeline, heatmap, treemap, or Sankey diagram (Munzner & Maguire, 2015). These patterns are effective because they rely on conventions that readers already understand. For example, rather than analysing the individual elements of a treemap, readers can quickly recognise it as a representation of hierarchical part-to-whole relationships.

The main practical difference is where the interpretive effort occurs. Grammar places the responsibility on the analyst, making each encoding decision explicit and easier to inspect, but potentially slower for unfamiliar readers to interpret. Idioms place more of that effort into established conventions, making them faster to understand but also creating a risk that the visual form implies relationships or meanings that the data does not actually support. Section 2.2 demonstrates this issue using the same data encoded in two different ways, resulting in contrasting conclusions.

2.2. Worked comparison: temporal data, the same calendar rows, two encodings

The Seattle dataset's calendar.csv does not contain a full year of repeated observations. Instead, it is a single forward-looking snapshot collected on 4 January 2016, showing each host's availability for the following 365

days. Two visualisations were created using these 1.39 million listing-night records.

Idiom-first reading. A conventional time-series visualisation, using a continuous weekly line with calendar time on the x-axis, shows the proportion of nights marked as unavailable throughout 2016. The rate decreases from 52% in January to 27% in December, a 25-percentage-point decline, with a noticeable increase during summer. Because the time-series idiom presents the data as a continuous progression over time, it naturally encourages a demand-side interpretation that the market became less constrained throughout the year.

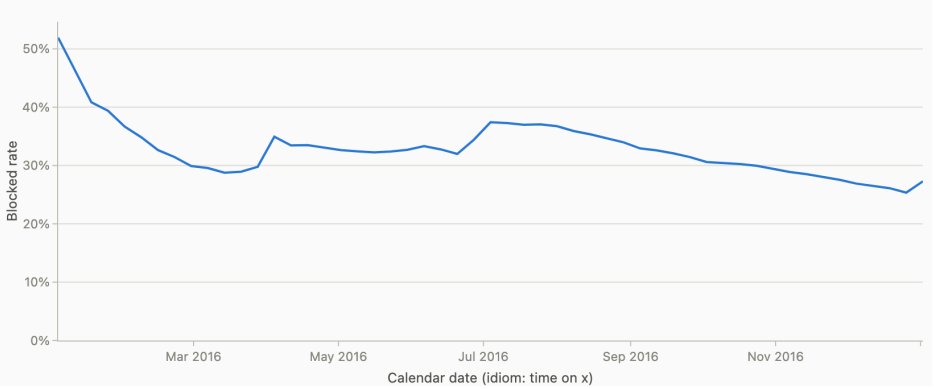

Figure 1. Idiom-first: blocked rate by calendar date

Grammar-first reading. Re-encoding the same data using the number of days before or after the scrape date, rather than the calendar date, changes the interpretation of the decline into a booking-accumulation curve. Nights close to the scrape date have had months to accumulate bookings and blocked dates, while nights further into the future have had little opportunity to do so. The Spearman correlation between booking horizon and blocked rate is −0.55. Therefore, the apparent “decline” is a measurement artefact caused by the way the data was collected and displayed on a time axis, making it appear to represent a genuine change over time.

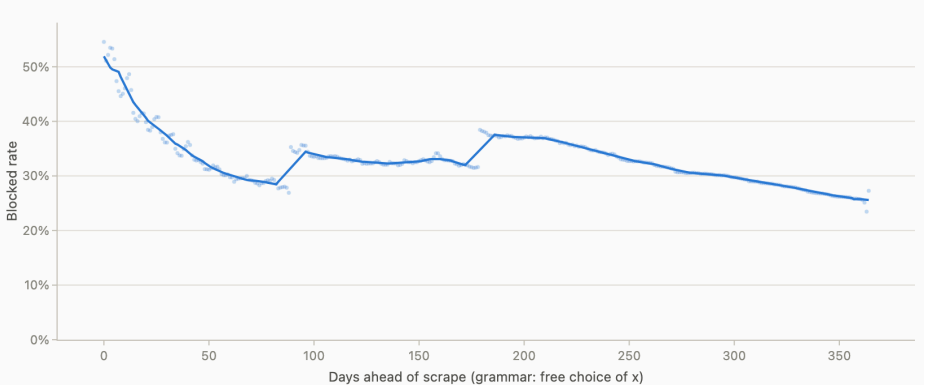

Figure 2. Grammar-first: blocked rate by booking horizon

This comparison clearly demonstrates the difference between the two approaches. The idiom-first visualisation, a familiar line chart, is mathematically correct, but its use of calendar time on the x-axis introduces an assumption that the grammar-first re-encoding reveals to be misleading. If a business dashboard relied only on the idiom-first chart, a reviewer could incorrectly conclude that the market was changing direction over time.

2.3. Worked comparison: spatial data, coordinate grammar vs the map idiom

The second dataset is based on the geolocation of the same listings, using latitude and longitude for 3,818 points across 87 neighbourhoods.

Idiom-first reading. The map idiom, with listing points displayed over street-level basemap tiles, is immediately understandable because readers can draw on existing geographic knowledge to identify areas such as Belltown and the waterfront. However, the basemap also introduces information that is not part of the dataset, including street names, water bodies, and political boundaries. In addition, the Web Mercator projection distorts area at northern latitudes, which can influence how readers visually compare areas.

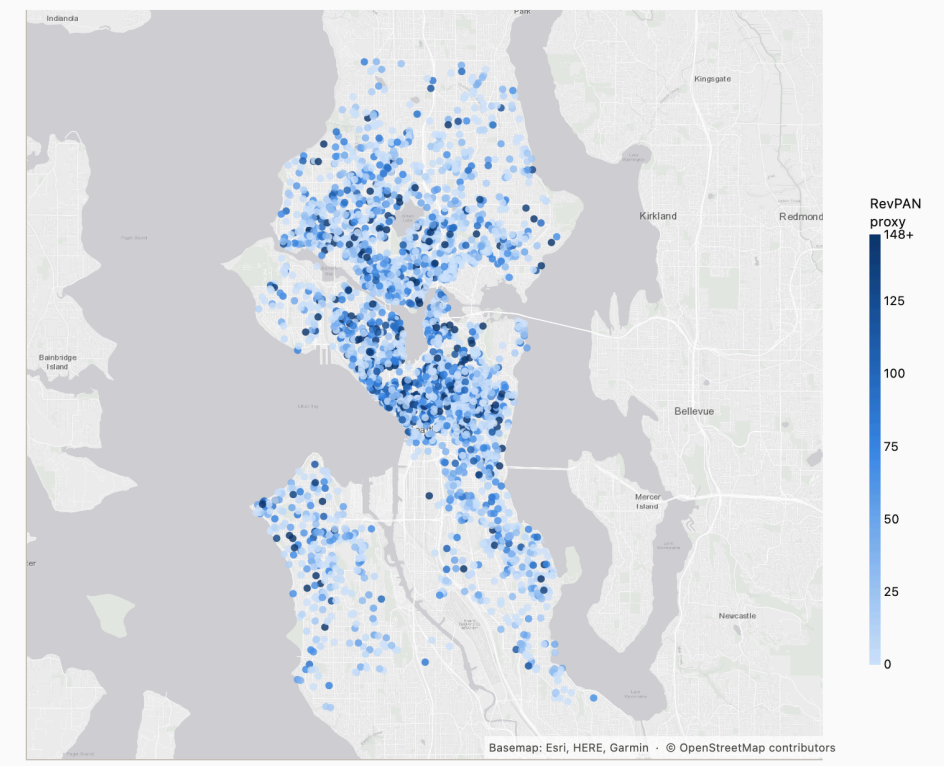

Figure 3. Idiom-first: listings over a basemap

Grammar-first reading. The same coordinates can instead be represented as a simple Cartesian scatter plot, using latitude and longitude as the position encodings and colour to represent a RevPAN proxy, without a basemap. This creates a purely geometric composition where the visual communicates only what the selected encoding channels specify. It is more difficult to interpret without prior geographic knowledge: a cluster of points has little meaning unless the reader knows which coordinates correspond to each neighbourhood, and areas that would normally be recognised as Puget Sound or Lake Washington appear only as unexplained gaps in the point distribution. However, this approach avoids introducing any information that is not present in the dataset.

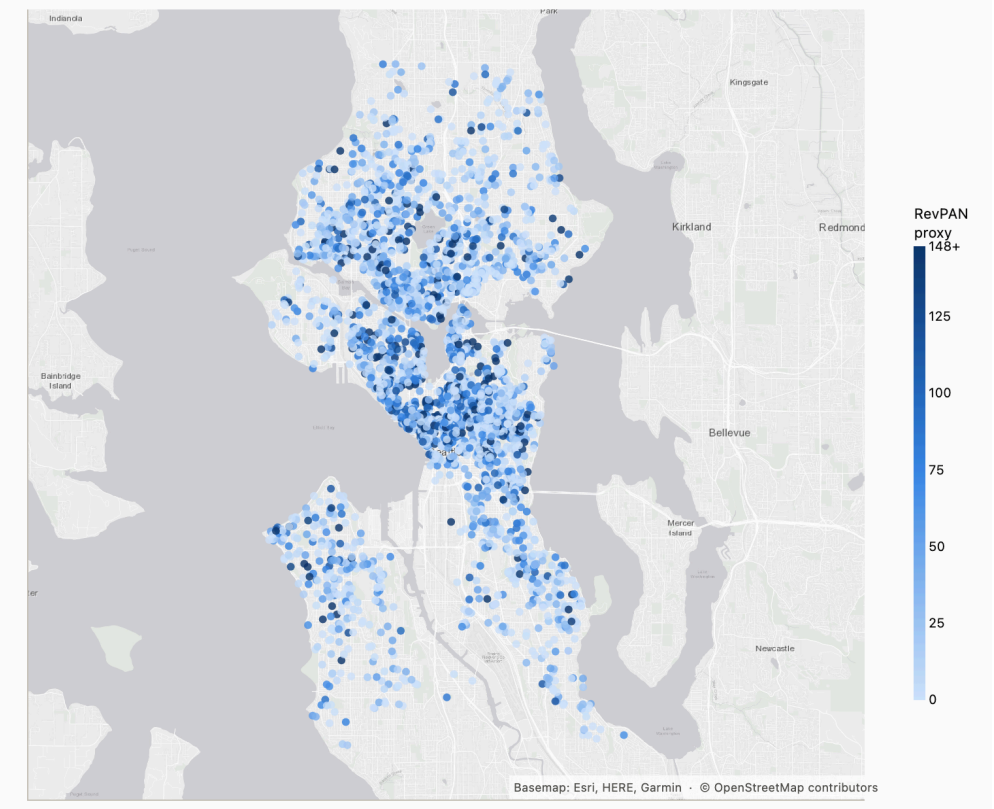

Figure 4. Grammar-first: listings as bare coordinates

The trade-off is straightforward: the idiom uses the reader's existing geographic knowledge to make interpretation faster, but this can introduce hidden distortions. The grammar-based version is more transparent about exactly what the data encodes, but it relies heavily on clear axis labels and legends for the reader to understand it.

2.4. Trade-offs: expressive power vs cognitive accessibility

The main strength of grammar-based visualisation is its flexibility to create new forms that fit the structure of a dataset. The horizon re-encoding in Section 2.2 was possible because the grammar treats the x-axis variable as an open choice rather than a fixed element of a particular chart type. Idioms, by contrast, sacrifice this flexibility for faster interpretation of familiar visual forms; for example, readers generally understand the purpose of a timeline without needing a legend.

However, the familiarity of an idiom can also become a weakness. Readers may automatically apply its conventional interpretation even when the underlying data does not meet the idiom's assumptions, as happened when a single scraped snapshot was interpreted as a time series. Munzner and Maguire (2015) frame this as a question of idiom effectiveness: the most appropriate idiom is not necessarily the most familiar one, but the one whose underlying assumptions best match the actual structure of the data.

2.5. Business bias: how businesses unintentionally mislead via grammar or idioms

Section 2.2 provides the main case study. A standard weekly line chart, despite using correctly calculated values and accurate labels, created a false narrative that supply was becoming less constrained. This happened because the chart's convention, where the x-axis represents calendar time, did not match the actual structure of the data, where the x-axis should represent the distance from a single scrape date.

The misleading interpretation was not caused by chart junk, deceptive scaling, or a selectively chosen axis. The problem came from the idiom itself. A business dashboard using this metric without the horizon check described in Section 2.2 could lead to incorrect pricing or inventory decisions based on a demand trend that does not actually exist. More broadly, this demonstrates the risk of choosing visual idioms based on familiarity and speed rather than their suitability for the dataset's collection method. Grammar-first exploration helps identify these mismatches before a visualisation is used in a stakeholder-facing dashboard.

3. Task 2: Multi-grain Data, Abstraction, and Semantic Layering

3.1. Domain and three data types

The domain is short-term rental hospitality e-commerce, using the Seattle Airbnb dataset introduced above. It generates three different data types without simulation:

- **Structured**: listings.csv and calendar.csv, fixed-schema tables, one row per listing or per listing-night, typed columns (price, room type, availability flag).
- **Semi-structured**: two fields embedded inside the structured listings table break its flat-schema assumption. amenities stores a brace-delimited, variable-length set per row (e.g. {TV,"Cable TV",Internet,"Wireless Internet",...}, averaging 14.5 items per listing but ranging from 0 to 30); host_verifications stores a Python-list-formatted field (e.g. ['email','phone','facebook','reviews','jumio'], averaging 4.6 per host). Neither has a fixed column count, so both were exploded into long-form tables (l0_amenity, l0_host_verification) before any aggregation. This semi-structured-to-structured conversion is the first abstraction step this task asks for.
- **Unstructured**: reviews.csv, 84,831 free-text guest comments, averaging 68.5 words each, carrying no fixed fields beyond a listing and reviewer ID. VADER sentiment scoring (Appendix A) converts this into a usable structured signal, but the source itself is prose.

3.2. Grain journey: atomic to aggregate to insight

The clearest single-table example is calendar availability, because its atomic grain is unambiguous in the dimensional-modelling sense (Kimball and Ross, 2013): one row is one listing, on one specific night: is this night available, and at what asking price. This atomic fact means nothing on its own; a single row (listing 1148517, 2016-03-14, available, $145) supports no decision. Value only appears once atomic rows are rolled up:

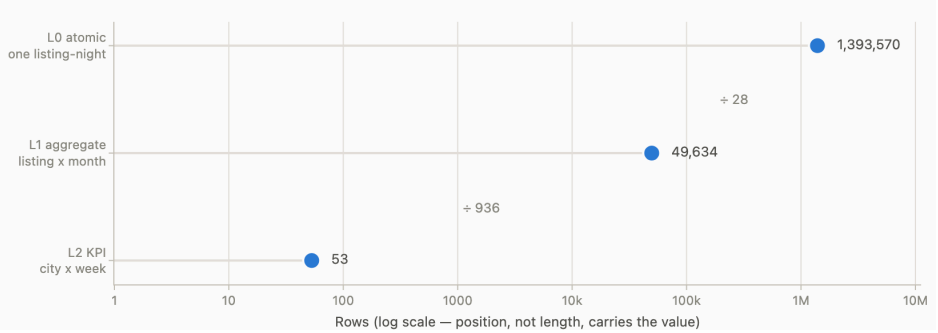

Figure 5. Row collapse, L0 to L2 (log scale)

- L0 to L1 (build_l1, data_access.rollup): 1,393,570 listing-nights collapse to 49,634 listing-month rows ("how did listing 1148517 perform in March 2016") by summing nights, blocked-nights and priced-nights, then deriving blocked_rate and asking_adr as ratios of those sums.
- L1 to L2 (build_l2): those roll up again to 53 market-week rows ("how is Seattle trending this week"), the level an executive dashboard actually reads.

The review table's grain journey runs in parallel: one review (l0_review) rolls to a listing-month sentiment mean (l1_review_month, 26,562 rows) and then to a neighbourhood sentiment_index (l2_kpi_neighbourhood, 87 rows). Two atomic sources, two independent grain journeys, converging at L2 into the same neighbourhood scorecard a decision-maker consults.

3.3. Semantic layering and the idiom that fits each level

Each layer calls for a different visualisation idiom because each layer answers a different question:

Layer	Question	Fitting idiom	Why
L0 atomic	"What happened on this night, for this listing?"	Raw table / detail-on-demand	1.39M rows have no meaningful chart form; only lookup matters
L1 aggregate	"How does this listing compare to others this month?"	Small multiples, scatter, sorted bar	Grammar-first: enough rows (49,634) for pattern-finding, too many for a single idiom to summarise safely

L2 KPI	"Is the market improving?"	Stat tile, choropleth, treemap	Idiom-first: 53–87 rows, one number needs to land in one glance for a non-analyst reader

Table 1. Semantic layering and the idiom that fits each level

This is the practical version of Task 1's grammar/idiom distinction: low-level data rewards grammar's flexibility (an analyst exploring L1 needs to try several encodings), while high-level KPIs reward idioms' speed (a manager reading L2 needs the treemap or stat-tile convention to work instantly, not a novel encoding they must first decode).

3.4. Critical review: where grain misalignment breaks the chain

Three specific problems emerged while building the semantic layer.

- **False positive caused by incorrect grain.** In calendar.csv, price is recorded only for nights marked as available and is null for blocked nights. An early version of the pipeline incorrectly combined price with blocked-night counts to create an “estimated revenue” KPI. This effectively multiplied an asking price from one group of nights by a booking count from a different group, producing a result that appeared reasonable but was invalid. The solution was to correct the data grain rather than the formula: revenue cannot be reliably derived from this source at any grain, so the column was removed instead of being included in the final output (Appendix A).
- **Over-generalisation caused by lost variation.** Across the city, the sentiment index has a mean of 0.892 and a standard deviation of 0.0245 between neighbourhoods, with values ranging from 0.812 to 0.946. Reducing this to a single KPI would simply indicate that “guests are happy” across the city. While technically accurate, this

is not useful for neighbourhood-level decisions because the aggregation hides meaningful differences.

- **Missed insight from a horizon-blind KPI.** The market-level blocked_rate KPI, when viewed without the L0 booking-horizon analysis developed for Task 1, creates the misleading conclusion that supply became 25 percentage points more available throughout 2016. The calculation itself is correct: it represents a genuine weekly average of a genuine daily availability flag. However, its apparent weekly grain does not match the dataset's actual collection structure, which is a single scrape date. The summer increase identified in Section 2.2/Act 4 is only visible when returning to L0 and examining the relationship between booking horizon and the metric rather than relying solely on the L2 KPI.

The common issue across all three cases is that semantic layering is only reliable when each aggregation is derived from the layer immediately below it, rather than repeatedly aggregating stored ratios. data_access.rollup enforces this structure. KPI grain must also be checked against the dataset's actual collection grain before the metric is presented to decision-makers.

4. Task 3: Designing Idiomatic, Multi-perspective Dashboards

4.1. The data and its three required components

Both dashboards read the same L1/L2 semantic layer built for Task 2, which carries all three required components natively: temporal (calendar date, booking horizon, review month), categorical (room type, property type, neighbourhood group, cancellation policy), and spatial (listing latitude/longitude, 87 neighbourhoods). No component was added artificially; all three were already present in the raw source and carried through the L0→L1→L2 pipeline.

4.2. Dashboard 1: exploratory (app/exploratory.py, port 8050)

The exploratory dashboard is user-driven: a filter row (neighbourhood group, room type, price, calendar window) sits above six coordinated views, and every chart reads one shared dcc.Store filter state, so narrowing a filter or drilling into a neighbourhood propagates to all views at once.

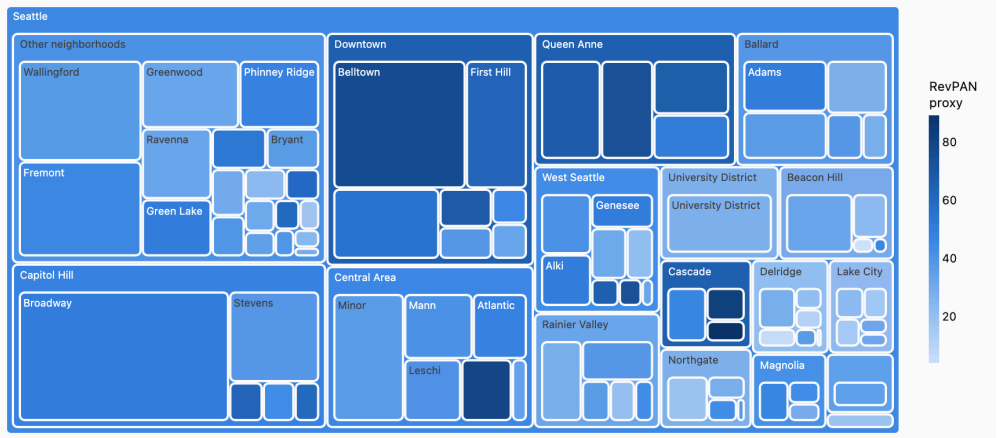

Figure 6. Treemap drill-down by neighbourhood

Drill-down: clicking any treemap tile or map point writes the same selection key, so the two spatial idioms (part-to-whole treemap, geospatial point map) act as one coordinated selector rather than two independent charts.

What-if analysis: a constant-elasticity repricing model lets the reader set a price change and a demand elasticity, then reads the projected RevPAN and blocked-rate response against the real baseline.

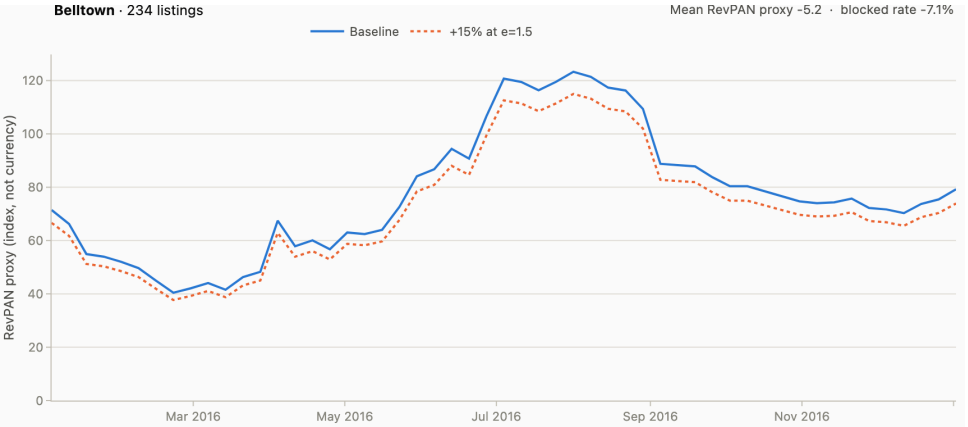

Figure 7. What-if scenario, Belltown

Filtering: five stat tiles, a sortable exact-value table, and continuous price/date-range sliders complete the exploratory set. Every visual claim has a table-view escape hatch, since the palette's aqua slot falls under 3:1 contrast on the light surface and needs that relief (Appendix A, palette validation). The whole dashboard follows Shneiderman's (1996) visual information-seeking mantra: overview (treemap, map) first, zoom and filter (the control row) next, details on demand (hover tooltips, the table view) last, rather than any single idiom trying to do all three jobs at once.

4.3. Dashboard 2: explanatory (app/explanatory.py, port 8051)

The explanatory dashboard is story-driven: no filters, a fixed seven-act scroll sequence, each act stating a claim, showing the one figure that tests it, and naming what the figure does not license. This is the martini-glass structure Segel and Heer (2010) describe for narrative visualization: author-driven acts followed by a closing, reader-controllable summary (the "So what" list). Acts 1–2 are the calendar-artifact reveal already developed in section 2.2 (Figures 1–2); acts 3–6 build the counter-evidence chain (review-based seasonality control, the summer-elevation exception, the flat weekday check, the price/turnover dumbbell). Act 7 adds a technique not used elsewhere in this report:

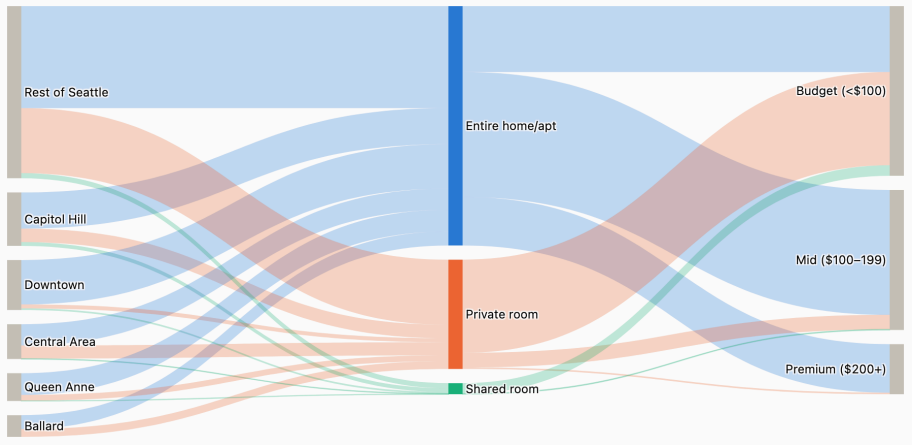

Figure 8. Sankey: group to room type to price tier

The Sankey resolves a question no single-panel chart in this report could answer directly: is price tier set by geography or by room type? Column-normalising the flow shows 97% of every premium ($200+) listing is an entire home, versus 3% private room and effectively none shared: a near-exclusive relationship. Row-normalising the other way, private- and shared-room listings land in the budget tier 85% and 96% of the time respectively, while entire-home listings alone reach premium at any real volume (20% of entire-home listings do). Geography barely perturbs this: the same room-type split recurs inside every one of the six neighbourhood-group nodes, visible in the diagram only because the Sankey's link colour is bound to room type rather than to source node. That is an encoding choice, not a visual accident.

4.4. Critical evaluation: grammar choices, idiom choices, and the decisions they shape

- **Grammar choices.** Both dashboards use the same scale and mark conventions, including a validated colour palette, a single categorical colour for each room type, and ratios recalculated from atomic totals at every aggregation level (Section 3.4). This consistency means readers can move between dashboards without having to relearn what each colour represents. The only intentional departure from this approach is the map-versus-bare-scatter comparison in Section 2.3. This was kept within Task 1 rather than included in the dashboards because dashboards need to support quick interpretation. The grammar-first bare scatter is therefore unsuitable for a decision-maker-facing tool because it requires geographic knowledge that the map idiom provides immediately.
- **Idiom choices and business consequences.** The treemap and Sankey diagram both represent composition, but they operate at different grains and answer different questions. The treemap shows where the inventory is, while the Sankey shows what the inventory consists of. Using the same idiom for both would have obscured one of these dimensions. Similarly, the what-if slider uses a scenario line alongside a baseline rather than displaying a single

projected value. This allows readers to see how the assumption affects the trend over time rather than focusing only on the endpoint. A single KPI-style projection might appear more authoritative, but it would hide the constant-elasticity assumption behind the calculation.

- **The central risk.** Acts 1–2 provide the clearest evidence for the brief's argument that grammar and idiom choices can influence business decisions. The same data, within the same dashboard family, produces opposing interpretations when encoded in two different ways. A pricing team relying only on the idiom-first chart in Act 1 could make inventory decisions based on a demand trend that the data does not actually support. The grammar-first version in Act 2 reveals the correct interpretation using the same underlying numbers. This demonstrates the risk of using an idiom whose conventions do not match the dataset's collection method. Visual polish alone would not have identified the problem; creating the grammar-first alternative exposed it.

5. Task 4: Future of Visualization Idioms with Emerging Tech

5.1. Two emerging technologies

Generative-AI narrative and NL-to-chart systems. LLM-driven tools now sit between the analyst and the grammar itself. LIDA (Dibia, 2023) generates chart specifications and accompanying narrative text from raw data with no analyst-authored encoding; Chat2VIS (Maddigan and Susnjak, 2023) converts a natural-language question directly into a Vega-Lite specification. Both replace the analyst's grammar-selection step with a model's inference of what encoding fits the question.

AR/VR immersive analytics. Marriott et al. (2018) and Fonnet and Prié (2021) survey a maturing field: data rendered in embodied 3D space rather than on a 2D screen, walking through a "data wall," or a VR digital twin of a physical site with live metrics overlaid on the geometry itself.

5.2. How they extend, disrupt, or complement grammar and idioms

Generative-AI tools disrupt the grammar layer: where Task 1 treated "choice of encoding" as the analyst's deliberate act, an LLM now makes that choice, collapsing the grammar/idiom distinction into a single opaque inference step the reader cannot inspect. They complement idiom literacy, though: Chat2VIS still outputs a conventional chart type a reader already recognises, so idiom-level familiarity is preserved even as grammar-level authorship is not.

AR/VR extends rather than disrupts: Figure 3's basemap idiom (Section 2.3) already borrows the reader's spatial prior knowledge to accelerate reading; a VR digital twin is that same idiom taken to its limit, adding embodied scale and viewpoint to the same underlying trade: familiarity accelerates interpretation, at the cost of everything the medium implies but the data does not state.

5.3. Case study: a conversational copilot on this dataset

Consider a hypothetical extension of the exploratory dashboard (Section 4.2): a host-ops team asks a Chat2VIS-style copilot, "why has our blocked rate fallen this year?" A model trained to answer fluently, without the horizon check developed in section 2.2, would plausibly generate exactly Act 1's chart (a correctly computed, idiomatically conventional line) and narrate it as a demand story. Nothing in the interaction would signal the error; the chart would be textbook-correct and the narration confident. Only a system explicitly prompted, or fine-tuned, to test collection-method artifacts, as the human analysis in Act 2 did, would surface the true finding. This is the same failure this report's Task 1 case study demonstrates, now delegated to a system with no incentive to doubt its first plausible answer.

5.4. Ethical and cognitive risks

- **Over-reliance on AI idioms.** AI-generated narratives can appear just as authoritative as human-written analysis because confident language is presented alongside polished visualisations. However,

they may lack the verification and critical checking expected from a careful analyst. Section 5.3 demonstrates this problem directly: the data issue identified across the earlier tasks is exactly the type of error a narrative-generation model may fail to detect by default.

- **Immersive bias.** The realism of a VR digital twin can create a stronger sense of certainty than the underlying data supports. For example, representing "occupancy" in a 3D environment using the blocked-rate proxy, which Appendix A identifies as an upper bound rather than a direct measurement, could make an estimate appear as reliable as an actual observation.
- **Interpretability.** An LLM-generated encoding choice or a black-box chart recommendation is less transparent and auditable than an explicit Vega-Lite specification. A decision-maker can see the resulting chart but may not be able to determine why that particular visualisation was selected or which assumptions influenced the choice.

5.5. Forward-looking: the next 5–10 years

The likely trajectory is not idioms disappearing but the locus of grammar choice moving from analyst to model, with idiom-level literacy remaining the stable interface between system and reader, because familiar chart forms will still be what makes an AI-generated visualization checkable at a glance. The defensible design pattern, and the one this report's explanatory dashboard already follows (Section 4.3, each act stating what its figure does not license), is generative tools that surface their assumptions alongside their output, not narratives presented as unqualified fact. Immersive and conversational interfaces will most likely supplement rather than replace the grammar-first dashboards built in Task 3, with exploratory, inspectable tools remaining the fallback whenever an idiom-first generated answer needs checking.

6. References

Dibia, V. (2023) ‘Lida: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models’, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 113–126. doi:10.18653/v1/2023.acl-demo.11.

Fonnet, A. and Prié, Y. (2021) ‘Survey of immersive analytics’, *IEEE Transactions on Visualization and Computer Graphics*, 27(3), pp. 2101–2122. doi:10.1109/tvcg.2019.2929033.

Kimball, R. and Ross, M. (2013) *The Data Warehouse Toolkit: The definitive guide to dimensional modeling*. Indianapolis, IN: John Wiley & Sons, Inc.

Maddigan, P. and Susnjak, T. (2023) ‘Chat2VIS: Generating data visualizations via natural language using CHATGPT, Codex and GPT-3 large language models’, *IEEE Access*, 11, pp. 45181–45193. doi:10.1109/access.2023.3274199.

Marriott, K., Schreiber, F., Dwyer, T., Klein, K., Riche, N.H., Itoh, T., Stuerzlinger, W. and Thomas, B.H. (eds.) (2018) *Immersive Analytics*. Cham: Springer.

Munzner, T. and Maguire, E. (2015) *Visualization analysis and design*. Boca Raton: CRC Press, Taylor & Francis Group.

Segel, E. and Heer, J. (2010) ‘Narrative visualization: Telling stories with data’, *IEEE Transactions on Visualization and Computer Graphics*, 16(6), pp. 1139–1148. doi:10.1109/tvcg.2010.179.

Shneiderman, B. (1996) ‘The eyes have it: A task by data type taxonomy for information visualizations’, *Proceedings 1996 IEEE Symposium on Visual Languages*, pp. 336–343. doi:10.1109/vl.1996.545307.

Wickham, H. (2010) ‘A layered grammar of graphics’, *Journal of Computational and Graphical Statistics*, 19(1), pp. 3–28. doi:10.1198/jcgs.2009.07098.

Wilkinson, L. (2005) *The grammar of graphics*. New York, NY: Springer Science+Business Media, Inc.

7. Appendix A: Data Lineage & Known Limitations

- A.0 Figure provenance
  - Figures 1-4: src/make_task1_figures.py (Figure 1 from data/l2_kpi_market.parquet; Figure 3 matches app/exploratory.py fig-map, basemap tiles composited server-side by src/staticmap.py)
  - Figure 5: src/make_task2_figures.py, counting rows in the parquet outputs of src/build_semantic_layer.py
  - Figures 6-8: src/make_task3_figures.py, rendered from app/exploratory.py and app/explanatory.py
- A.1 Semantic layer tables
  - L0 atomic
    - l0_listing: one row per listing; money and percent strings cast to numeric
    - l0_calendar: one row per listing-night; available 't'/'f' -> is_booked bool
    - l0_review: one row per review; VADER compound sentiment, English-only
    - l0_amenity: amenities set exploded to one row per (listing, amenity)
    - l0_host_verification: host_verifications list exploded per (host, method)
  - L1 aggregate
    - l1_listing_month: listing x month: blocked_rate, asking_adr, revpan_proxy
    - l1_neighbourhood_week: neighbourhood x week: blocked_rate, asking_adr, revpan_proxy
    - l1_review_month: listing x month: review volume, mean sentiment
  - L2 kpi
    - l2_kpi_neighbourhood: supply + demand + voice per neighbourhood
    - l2_kpi_market: city-wide weekly blocked_rate / asking_adr / revpan_proxy

        - l2_listing_scorecard: per-listing KPI roll-up for ranking and drill-down
        - l2_amenity_penetration: amenity share of listings by neighbourhood group
- A.2 Known limitations
    - calendar.price is populated ONLY on available=='t' nights (934,542) and is null on all 459,028 unavailable nights -- it is an asking price on open nights, never a transacted rate. Realised revenue is NOT derivable; no revenue column is emitted.
    - calendar 'f' conflates guest-booked with host-blocked, so blocked_rate is an UPPER BOUND on true occupancy, not a measurement of it
    - revpan_proxy = asking_adr * blocked_rate is a comparative index, not a currency amount
    - asking_adr is null for 13,519 of 49,634 listing-months where the listing was blocked all month and quoted no price
    - review dates (2009-06-07..2016-01-03) do not overlap calendar (2016-01-04..2017-01-02)
    - review velocity is a demand proxy: only a minority of stays leave a review
    - VADER is English-only and lexicon-based; non-English reviews are nulled, not translated
    - listing attributes are a single 2016-01-04 scrape, so they are not time-varying

8. Appendix B: Source Code

All code is original and was written for this assessment. Python 3.14; dependencies are pinned in requirements.txt (dash 4.4.1, pandas 3.0.5, numpy 2.5.2, pyarrow 25.0.1, plotly 7.0.0, kaleido 1.3.0, requests 2.34.2, vaderSentiment 3.3.2).

Run order: `python src/build_semantic_layer.py` (writes data/*.parquet), then the three figure scripts, then either dashboard.

- B.0 Reproduction

  The three raw CSVs (listings.csv, calendar.csv, reviews.csv) sit at the project root, alongside src/ and app/. None of the pinned dependencies ship with a stock Python install, so build an isolated environment first and run every script through that interpreter; invoking a bare system `python3` fails at the first import with `ModuleNotFoundError: No module named 'pandas'`.

  —

  python3 -m venv .venv

  .venv/bin/python -m pip install -r requirements.txt

  —

  Then run the scripts in this order -- each writes into the directory the next one reads:

  —

  .venv/bin/python src/build_semantic_layer.py # raw CSVs -> data/*.parquet + data/lineage.json

  .venv/bin/python src/make_task1_figures.py # -> figures/fig1..fig4

  .venv/bin/python src/make_task2_figures.py # -> figures/fig5

  .venv/bin/python src/make_task3_figures.py # -> figures/fig6..fig8

  .venv/bin/python app/exploratory.py # -> http://127.0.0.1:8050

  .venv/bin/python app/explanatory.py # -> http://127.0.0.1:8051

  —

  build_semantic_layer.py is the only prerequisite for everything downstream: it scores sentiment on 84,831 reviews and takes roughly a minute. The figure scripts and the dashboards all read the Parquet tables it emits, so the figures are not a prerequisite for the dashboards. The two

dashboards are servers, each holding its terminal until stopped with Ctrl-C, so run them one at a time or in separate terminals.

Two environment-dependent steps are worth naming. make_task1_figures.py fetches basemap tiles over the network for Figure 3 and caches them in .tilecache/, so it needs an internet connection on first run only. PNG export goes through kaleido, which drives a headless Chrome; if no Chrome is present the export raises, and `.venv/bin/plotly_get_chrome` installs a private copy. On Windows the interpreter path is .venv\Scripts\python.

- B.1 src/build_semantic_layer.py

  Builds the L0/L1/L2 semantic layer from the raw CSVs.

```
"""
Build the three-tier semantic layer for CIS6027 WRIT1.


   L0  atomic grain      one listing / one calendar-night / one
review / one amenity
   L1  aggregates        listing-month, neighbourhood-week,
review-month
   L2  KPIs              neighbourhood scorecards, market
time-series, listing scorecards


Source: Seattle Airbnb Open Data (Kaggle: airbnb/seattle).
Outputs Parquet to data/ so dashboard callbacks read pre-computed
frames.


Run:  .venv/bin/python src/build_semantic_layer.py
"""


import ast
import csv
import io
import json
```

```
import re
from pathlib import Path


import pandas as pd
from vaderSentiment.vaderSentiment import
SentimentIntensityAnalyzer


ROOT = Path(__file__).resolve().parent.parent
RAW = ROOT
OUT = ROOT / "data"
OUT.mkdir(exist_ok=True)


MONEY = re.compile(r"[^0-9.]")



def money(s: pd.Series) -> pd.Series:
    """'$1,250.00' -> 1250.0"""
    return pd.to_numeric(s.astype("string").str.replace(MONEY, "",
regex=True), errors="coerce")



def pct(s: pd.Series) -> pd.Series:
    """'96%' -> 0.96"""
    return pd.to_numeric(s.astype("string").str.rstrip("%"),
errors="coerce") / 100.0



def parse_amenities(cell) -> list[str]:
    """'{TV,"Cable TV",Internet}' -> ['TV', 'Cable TV', 'Internet']

    Brace-delimited set with quoted multi-word members: the
semi-structured
    payload embedded inside an otherwise flat CSV cell.
    """
```

```
    if not isinstance(cell, str):
        return []
    inner = cell.strip().lstrip("{").rstrip("}")
    if not inner:
        return []
    row = next(csv.reader(io.StringIO(inner)), [])
    return [a.strip().strip('"') for a in row if
a.strip().strip('"')]



def parse_verifications(cell) -> list[str]:
    """['email', 'phone']" -> ['email', 'phone']"""
    if not isinstance(cell, str):
        return []
    try:
        val = ast.literal_eval(cell)
    except (ValueError, SyntaxError):
        return []
    return [str(v) for v in val] if isinstance(val, (list, tuple))
else []



# ---------------------------------------------------------------
L0: atomic


def build_l0():
    print("L0  reading raw files ...")
    listings = pd.read_csv(RAW / "listings.csv", low_memory=False)
    reviews = pd.read_csv(RAW / "reviews.csv")
    calendar = pd.read_csv(RAW / "calendar.csv")

    # --- l0_listing
-----------------------------------------------------
    keep = {
```

```
    "id": "listing_id",
    "host_id": "host_id",
    "host_since": "host_since",
    "host_is_superhost": "host_is_superhost",
    "host_response_rate": "host_response_rate",
    "host_acceptance_rate": "host_acceptance_rate",
    "neighbourhood_cleansed": "neighbourhood",
    "neighbourhood_group_cleansed": "neighbourhood_group",
    "latitude": "latitude",
    "longitude": "longitude",
    "property_type": "property_type",
    "room_type": "room_type",
    "accommodates": "accommodates",
    "bathrooms": "bathrooms",
    "bedrooms": "bedrooms",
    "beds": "beds",
    "price": "price",
    "cleaning_fee": "cleaning_fee",
    "security_deposit": "security_deposit",
    "minimum_nights": "minimum_nights",
    "maximum_nights": "maximum_nights",
    "availability_365": "availability_365",
    "number_of_reviews": "number_of_reviews",
    "review_scores_rating": "review_scores_rating",
    "review_scores_cleanliness": "review_scores_cleanliness",
    "review_scores_location": "review_scores_location",
    "review_scores_value": "review_scores_value",
    "reviews_per_month": "reviews_per_month",
    "instant_bookable": "instant_bookable",
    "cancellation_policy": "cancellation_policy",
}
lst = listings[list(keep)].rename(columns=keep).copy()
```

```
    for col in ("price", "cleaning_fee", "security_deposit"):
        lst[col] = money(lst[col])
    for col in ("host_response_rate", "host_acceptance_rate"):
        lst[col] = pct(lst[col])
    for col in ("host_is_superhost", "instant_bookable"):
        lst[col] = lst[col].map({"t": True, "f": False})
    lst["host_since"] = pd.to_datetime(lst["host_since"],
errors="coerce")


    lst.to_parquet(OUT / "10_listing.parquet", index=False)


    # --- 10_amenity / 10_host_verification
------------------------------
    # Semi-structured -> normalised long form. This explode IS the
abstraction
    # step Task 2 asks you to trace, so it stays a visible
artefact.
    am = listings[["id", "amenities"]].copy()
    am["amenity"] = am["amenities"].map(parse_amenities)
    am = am.explode("amenity").dropna(subset=["amenity"])
    am = am[["id", "amenity"]].rename(columns={"id": "listing_id"})
    am.to_parquet(OUT / "10_amenity.parquet", index=False)


    hv = listings[["host_id",
"host_verifications"]].drop_duplicates("host_id").copy()
    hv["verification"] =
hv["host_verifications"].map(parse_verifications)
    hv = hv.explode("verification").dropna(subset=["verification"])
    hv = hv[["host_id", "verification"]]
    hv.to_parquet(OUT / "10_host_verification.parquet",
index=False)


    # --- 10_calendar
-----------------------------------------------------
```

```
    cal = calendar.rename(columns={"listing_id":
"listing_id"}).copy()
    cal["date"] = pd.to_datetime(cal["date"])
    cal["price"] = money(cal["price"])
    # 'f' = not available. NOTE: this conflates guest-booked with
host-blocked.
    # The distinction is unrecoverable from this source; occupancy
below is
    # therefore an upper bound. Task 2's grain-mismatch critique
starts here.
    cal["is_booked"] = cal["available"].eq("f")
    cal["year"] = cal["date"].dt.year
    cal["month"] = cal["date"].dt.to_period("M").astype(str)
    cal["week"] = cal["date"].dt.to_period("W").dt.start_time
    cal["dow"] = cal["date"].dt.dayofweek
    cal["is_weekend"] = cal["dow"].isin([4, 5])
    cal = cal[["listing_id", "date", "year", "month", "week",
"dow",
               "is_weekend", "is_booked", "price"]]
    cal.to_parquet(OUT / "10_calendar.parquet", index=False)


    # --- 10_review
------------------------------------------------------------
    rev = reviews.rename(columns={"id": "review_id"}).copy()
    rev["date"] = pd.to_datetime(rev["date"])
    rev = rev.dropna(subset=["comments"])
    rev["comments"] = rev["comments"].astype("string")
    rev["char_len"] = rev["comments"].str.len()
    rev["word_count"] = rev["comments"].str.split().str.len()
    # Crude language flag: VADER is English-only, so scoring
non-English text
    # produces noise near 0. Flag it rather than silently averaging
it in.
    ascii_ratio = rev["comments"].map(lambda s: sum(c.isascii() for
c in s) / max(len(s), 1))
```

```
    rev["likely_english"] = ascii_ratio > 0.95


    print(f"L0  scoring sentiment on {len(rev):,} reviews ...")
    sia = SentimentIntensityAnalyzer()
    rev["sentiment"] = [sia.polarity_scores(t)["compound"] for t in
rev["comments"]]
    rev.loc[~rev["likely_english"], "sentiment"] = pd.NA


    rev["month"] = rev["date"].dt.to_period("M").astype(str)
    rev["week"] = rev["date"].dt.to_period("W").dt.start_time
    rev = rev[["review_id", "listing_id", "date", "month", "week",
"reviewer_id",
               "comments", "char_len", "word_count",
"likely_english", "sentiment"]]
    rev.to_parquet(OUT / "l0_review.parquet", index=False)


    print(f"L0  listing={len(lst):,}  calendar={len(cal):,}
review={len(rev):,} "
          f" amenity={len(am):,}  host_verification={len(hv):,}")
    return lst, cal, rev, am




# ------------------------------------------------------------ L1:
aggregates


def build_l1(lst, cal, rev):
    """
    IMPORTANT — what `price` actually means in this source.


    calendar.price is populated ONLY on nights where available ==
't'
    (934,542 rows) and is null on every available == 'f' night
(459,028 rows,
    zero prices). So the price series is the *asking price on open
nights*,
```

```
    never a transacted rate on a sold night.


    Two consequences, both load-bearing:

      1. Realised revenue is not derivable from this dataset.
Anything of the

         form price x booked_nights multiplies an open-night asking
price by a

         blocked-night count -- two disjoint sets of nights. No
such column is

         emitted here.

      2. 'occupancy' is not measurable either. available == 'f'
conflates

         guest-booked with host-blocked, so the honest name is
`blocked_rate`

         and it is an UPPER BOUND on true occupancy.


    Metrics are therefore named for what they measure:
`asking_adr`,

    `blocked_rate`, and `revpan_proxy` (their product -- a
comparative index,

    not a currency amount). All aggregates roll up from atomic sums
so that

    revpan_proxy == asking_adr * blocked_rate holds exactly at
every level;

    mean-of-means would break that identity wherever price and
availability

    correlate.

    """

    print("L1  aggregating ...")

    geo = lst[["listing_id", "neighbourhood",
"neighbourhood_group",

               "room_type", "property_type", "latitude",
"longitude"]]


    # --- listing x month
-------------------------------------------------
```

```
    lm = (cal.groupby(["listing_id", "month"], observed=True)
              .agg(nights=("date", "size"),
                   blocked_nights=("is_booked", "sum"),
                   price_sum=("price", "sum"),
                   open_nights=("price", "count"),
                   price_min=("price", "min"),
                   price_max=("price", "max"))
              .reset_index())
    lm["blocked_rate"] = lm["blocked_nights"] / lm["nights"]
    # asking_adr is NaN where a listing was blocked for the whole
month and so
    # quoted no price at all -- 13,519 of 49,634 listing-months.
    lm["asking_adr"] = lm["price_sum"] /
lm["open_nights"].replace(0, pd.NA)
    lm["revpan_proxy"] = lm["asking_adr"] * lm["blocked_rate"]
    lm = lm.merge(geo, on="listing_id", how="left")
    lm.to_parquet(OUT / "11_listing_month.parquet", index=False)


    # --- neighbourhood x week
-----------------------------------------
    cw = cal.merge(geo, on="listing_id", how="left")
    nw = (cw.groupby(["neighbourhood_group", "neighbourhood",
"week"], observed=True)
              .agg(listings=("listing_id", "nunique"),
                   nights=("date", "size"),
                   blocked_nights=("is_booked", "sum"),
                   price_sum=("price", "sum"),
                   open_nights=("price", "count"))
              .reset_index())
    nw["blocked_rate"] = nw["blocked_nights"] / nw["nights"]
    nw["asking_adr"] = nw["price_sum"] /
nw["open_nights"].replace(0, pd.NA)
    nw["revpan_proxy"] = nw["asking_adr"] * nw["blocked_rate"]
```

```
    nw.to_parquet(OUT / "l1_neighbourhood_week.parquet",
index=False)

    # --- review volume / sentiment x listing x month
----------------------
    rm = (rev.groupby(["listing_id", "month"], observed=True)
            .agg(reviews=("review_id", "size"),
                 sentiment_mean=("sentiment", "mean"),
                 words_mean=("word_count", "mean"),
                 english_share=("likely_english", "mean"))
            .reset_index()
            .merge(geo, on="listing_id", how="left"))
    rm.to_parquet(OUT / "l1_review_month.parquet", index=False)

    print(f"L1  listing_month={len(lm):,}
neighbourhood_week={len(nw):,}  "
          f"review_month={len(rm):,}")
    return lm, nw, rm



#
------------------------------------------------------------------
L2: KPIs


def build_l2(lst, lm, nw, rm, rev, am):
    print("L2  computing KPIs ...")

    # --- neighbourhood scorecard
---------------------------------------
    supply = (lst.groupby(["neighbourhood_group", "neighbourhood"],
observed=True)
                 .agg(listings=("listing_id", "nunique"),
                      hosts=("host_id", "nunique"),
                      median_price=("price", "median"),
                      mean_rating=("review_scores_rating", "mean"),
```

```
                    superhost_share=("host_is_superhost",
"mean"),
                    lat=("latitude", "mean"),
                    lon=("longitude", "mean"))
                .reset_index())

    # Roll up from atomic sums, not from means, so the identity
holds.
    demand = (lm.groupby(["neighbourhood_group", "neighbourhood"],
observed=True)
                .agg(nights=("nights", "sum"),
                     blocked_nights=("blocked_nights", "sum"),
                     price_sum=("price_sum", "sum"),
                     open_nights=("open_nights", "sum"))
                .reset_index())
    demand["blocked_rate"] = demand["blocked_nights"] /
demand["nights"]
    demand["asking_adr"] = demand["price_sum"] /
demand["open_nights"].replace(0, pd.NA)
    demand["revpan_proxy"] = demand["asking_adr"] *
demand["blocked_rate"]
    demand = demand.drop(columns=["price_sum"])

    # Review velocity: reviews per listing per month over the
review window.
    # A demand proxy, not a booking count -- only a minority of
stays review.
    window_months = rev["date"].dt.to_period("M").nunique()
    voice = (rev.merge(lst[["listing_id", "neighbourhood_group",
"neighbourhood"]],
                       on="listing_id", how="left")
                .groupby(["neighbourhood_group", "neighbourhood"],
observed=True)
                .agg(reviews_total=("review_id", "size"),
                     sentiment_index=("sentiment", "mean"),
                     english_share=("likely_english", "mean"))
```

```
                .reset_index())


    kpi = supply.merge(demand, on=["neighbourhood_group",
"neighbourhood"], how="left")
    kpi = kpi.merge(voice, on=["neighbourhood_group",
"neighbourhood"], how="left")
    kpi["review_velocity"] = kpi["reviews_total"] / kpi["listings"]
/ window_months
    kpi.to_parquet(OUT / "12_kpi_neighbourhood.parquet",
index=False)


    # --- market-level weekly time series
--------------------------------
    market = (nw.groupby("week", observed=True)
                .agg(listings=("listings", "sum"),
                     nights=("nights", "sum"),
                     blocked_nights=("blocked_nights", "sum"),
                     price_sum=("price_sum", "sum"),
                     open_nights=("open_nights", "sum"))
                .reset_index())
    market["blocked_rate"] = market["blocked_nights"] /
market["nights"]
    market["asking_adr"] = market["price_sum"] /
market["open_nights"].replace(0, pd.NA)
    market["revpan_proxy"] = market["asking_adr"] *
market["blocked_rate"]
    market.to_parquet(OUT / "12_kpi_market.parquet", index=False)


    # --- listing scorecard
-----------------------------------------------
    ls = (lm.groupby("listing_id", observed=True)
            .agg(nights=("nights", "sum"),
                 blocked_nights=("blocked_nights", "sum"),
                 price_sum=("price_sum", "sum"),
                 open_nights=("open_nights", "sum"))
```

```
            .reset_index())
    ls["blocked_rate"] = ls["blocked_nights"] / ls["nights"]
    ls["asking_adr"] = ls["price_sum"] /
ls["open_nights"].replace(0, pd.NA)
    ls["revpan_proxy"] = ls["asking_adr"] * ls["blocked_rate"]
    ls = ls.drop(columns=["price_sum"])


    rs = (rm.groupby("listing_id", observed=True)
            .agg(reviews=("reviews", "sum"),
                 sentiment=("sentiment_mean", "mean"))
            .reset_index())
    amn = am.groupby("listing_id",
observed=True).size().rename("amenity_count").reset_index()
    card = (lst.merge(ls, on="listing_id", how="left")
               .merge(rs, on="listing_id", how="left")
               .merge(amn, on="listing_id", how="left"))
    card["amenity_count"] =
card["amenity_count"].fillna(0).astype(int)
    card.to_parquet(OUT / "12_listing_scorecard.parquet",
index=False)


    # --- amenity penetration
-------------------------------------------
    pen = (am.merge(lst[["listing_id", "neighbourhood_group",
"room_type"]],
                    on="listing_id", how="left")
             .groupby(["amenity", "neighbourhood_group"],
observed=True)
             .size().rename("listings").reset_index())
    totals = lst.groupby("neighbourhood_group",
observed=True).size().rename("total").reset_index()
    pen = pen.merge(totals, on="neighbourhood_group", how="left")
    pen["penetration"] = pen["listings"] / pen["total"]
    pen.to_parquet(OUT / "12_amenity_penetration.parquet",
index=False)
```

```
   print(f"L2  kpi_neighbourhood={len(kpi):,}
kpi_market={len(market):,}  "
         f"listing_scorecard={len(card):,}
amenity_penetration={len(pen):,}")
   return kpi, market, card



#
-----------------------------------------------------------------
lineage


def write_lineage():
   lineage = {
       "source": "Seattle Airbnb Open Data (Kaggle:
airbnb/seattle)",
       "layers": {
           "L0_atomic": {
               "l0_listing": "one row per listing; money and
percent strings cast to numeric",
               "l0_calendar": "one row per listing-night;
available 't'/'f' -> is_booked bool",
               "l0_review": "one row per review; VADER compound
sentiment, English-only",
               "l0_amenity": "amenities set exploded to one row
per (listing, amenity)",
               "l0_host_verification": "host_verifications list
exploded per (host, method)",
           },
           "L1_aggregate": {
               "l1_listing_month": "listing x month: blocked_rate,
asking_adr, revpan_proxy",
               "l1_neighbourhood_week": "neighbourhood x week:
blocked_rate, asking_adr, revpan_proxy",
               "l1_review_month": "listing x month: review volume,
mean sentiment",
```

```
            },
            "L2_kpi": {
                "l2_kpi_neighbourhood": "supply + demand + voice
per neighbourhood",
                "l2_kpi_market": "city-wide weekly blocked_rate /
asking_adr / revpan_proxy",
                "l2_listing_scorecard": "per-listing KPI roll-up
for ranking and drill-down",
                "l2_amenity_penetration": "amenity share of
listings by neighbourhood group",
            },
        },
        "known_limitations": [
            "calendar.price is populated ONLY on available=='t'
nights (934,542) and is null on all 459,028 unavailable nights --
it is an asking price on open nights, never a transacted rate.
Realised revenue is NOT derivable; no revenue column is emitted.",
            "calendar 'f' conflates guest-booked with host-blocked,
so blocked_rate is an UPPER BOUND on true occupancy, not a
measurement of it",
            "revpan_proxy = asking_adr * blocked_rate is a
comparative index, not a currency amount",
            "asking_adr is null for 13,519 of 49,634 listing-months
where the listing was blocked all month and quoted no price",
            "review dates (2009-06-07..2016-01-03) do not overlap
calendar (2016-01-04..2017-01-02)",
            "review velocity is a demand proxy: only a minority of
stays leave a review",
            "VADER is English-only and lexicon-based; non-English
reviews are nulled, not translated",
            "listing attributes are a single 2016-01-04 scrape, so
they are not time-varying",
        ],
    }
    (OUT / "lineage.json").write_text(json.dumps(lineage,
indent=2))
```

```
if __name__ == "__main__":
    lst, cal, rev, am = build_l0()
    lm, nw, rm = build_l1(lst, cal, rev)
    build_l2(lst, lm, nw, rm, rev, am)
    write_lineage()
    print("
done ->", OUT)
```

- B.2 src/make_task1_figures.py

  Renders Figures 1-4.

```
"""
Report figures for Task 1 -- grammar vs idiom, temporal and
spatial.

Reuses the same data the dashboards read (app/data_access.py,
app/story_data.py)
so every figure here is consistent with what the dashboards show;
nothing is
recomputed differently for the report.

One deliberate exception, figure 3. The dashboard draws its map
through
MapLibre/WebGL, which Kaleido's headless Chrome cannot rasterise
-- exporting it
yields an empty canvas, which is what the previous
fig3_idiom_map.png was.
Figure 3 therefore composites raster tiles server-side
(src/staticmap.py) and
draws the points as an ordinary Cartesian scatter in Web-Mercator
coordinates:
same projection, same ramp, an equivalent light-grey canvas
basemap, no GPU.

Run:  .venv/bin/python src/make_task1_figures.py
"""
```

```
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "app"))
sys.path.insert(0, str(ROOT / "src"))

import plotly.graph_objects as go

import data_access as da
import staticmap as sm
import story_data as sd
from theme import CATEGORICAL, CHROME, SEQUENTIAL, tpl

OUT = ROOT / "figures"
OUT.mkdir(exist_ok=True)
MODE = "light"
SEQ = [[i / (len(SEQUENTIAL) - 1), h] for i, h in
enumerate(SEQUENTIAL)]
c, blue = CHROME[MODE], CATEGORICAL[MODE][0]



def save(fig, name, w=820, h=340):
    fig.write_image(str(OUT / name), width=w, height=h, scale=2)
    print("wrote", name)



# ---- Fig 1: idiom-first temporal (weekly line, calendar time on
x) --------
g = da.kpi_market().sort_values("week")
f1 = go.Figure()
f1.add_trace(go.Scatter(x=g["week"], y=g["blocked_rate"],
mode="lines",
```

```
                      line=dict(width=2, color=blue),
                      hovertemplate="%{x|%d %b %Y}<br>Blocked
%{y:.1%}<extra></extra>"))
f1.update_layout(template=tpl(MODE), showlegend=False,
                margin=dict(l=8, r=8, t=20, b=8),
                yaxis=dict(tickformat=".0%", rangemode="tozero",
title="Blocked rate"),
                xaxis=dict(showgrid=False, title="Calendar date
(idiom: time on x)"))
save(f1, "fig1_idiom_calendar.png")


# ---- Fig 2: grammar-first temporal (horizon on x)
--------------------------
h = sd.horizon_curve()
f2 = go.Figure()
f2.add_trace(go.Scatter(x=h["horizon"], y=h["blocked_rate"],
mode="markers",
                      marker=dict(size=3.5, color=blue,
opacity=0.28), hoverinfo="skip"))
f2.add_trace(go.Scatter(x=h["horizon"], y=h["blocked_smooth"],
mode="lines",
                      line=dict(width=2, color=blue),
                      hovertemplate="%{x} days ahead<br>Blocked
%{y:.1%}<extra></extra>"))
f2.update_layout(template=tpl(MODE), showlegend=False,
                margin=dict(l=8, r=8, t=20, b=8),
                yaxis=dict(tickformat=".0%", rangemode="tozero",
title="Blocked rate"),
                xaxis=dict(showgrid=False,
                           title="Days ahead of scrape (grammar:
free choice of x)"))
save(f2, "fig2_grammar_calendar.png")


# ---- shared spatial encoding
-------------------------------------------
```

```
# RevPAN proxy is heavily right-skewed (median 17, p95 148, max
517). On a
# linear 0-max ramp the median listing lands at 3% of the scale
and the whole
# city reads as one flat pale wash, in BOTH figures. Capping at
p95 and saying
# so on the colourbar keeps the encoding honest while making it
legible; the
# cap is identical in figures 3 and 4 so the idiom/grammar
comparison is still
# a like-for-like one.
lst = da.listings().dropna(subset=["latitude", "longitude",
"revpan_proxy"])
CMAX = float(lst["revpan_proxy"].quantile(0.95))
CBAR = dict(title=dict(text="RevPAN<br>proxy",
font=dict(size=11)),
            thickness=9, len=0.66, y=0.44, yanchor="middle",
            outlinewidth=0, tickfont=dict(size=10),
            tickvals=[0, 25, 50, 75, 100, 125, CMAX],
            ticktext=["0", "25", "50", "75", "100", "125",
f"{CMAX:.0f}+"])


MAP_W, MAP_H = 760, 620
PLOT_W = MAP_W - 120                      # colourbar + margins
take the rest


# ---- Fig 3: idiom-first spatial (map idiom, basemap tiles)
-----------------
lon_r, lat_r = sm.fit_bounds(lst["longitude"], lst["latitude"],
                             pad=0.08, aspect=(MAP_H - 16) /
PLOT_W)
f3 = go.Figure()
f3.update_layout(template=tpl(MODE), showlegend=False,
                 margin=dict(l=8, r=8, t=8, b=8))
project = sm.basemap(f3, lon_r, lat_r, device_px=PLOT_W * 2,
                     style=CHROME[MODE]["map_style"])
```

```
mx, my = project(lst["longitude"], lst["latitude"])
f3.add_trace(go.Scatter(
   x=mx, y=my, mode="markers", customdata=lst[["revpan_proxy"]],
   marker=dict(size=6, color=lst["revpan_proxy"], colorscale=SEQ,
               cmin=0, cmax=CMAX, opacity=0.82,
line=dict(width=0),
               colorbar=CBAR),
   hovertemplate="RevPAN proxy
%{customdata[0]:,.1f}<extra></extra>"))
save(f3, "fig3_idiom_map.png", w=MAP_W, h=MAP_H)


# ---- Fig 4: grammar-first spatial (bare Cartesian scatter, no
basemap) -----
# Position = lat/long as plain x/y encoding channels. Nothing
beyond the four
# stated channels (x, y, colour, and the axis scales themselves)
is asserted --
# no basemap, no implied streets or boundaries, no Mercator area
distortion.
# The plot box is sized to the data so the equal-scale constraint
does not
# leave two thirds of the panel empty.
f4 = go.Figure()
f4.add_trace(go.Scatter(
   x=lst["longitude"], y=lst["latitude"], mode="markers",
   marker=dict(size=6, color=lst["revpan_proxy"], colorscale=SEQ,
               cmin=0, cmax=CMAX, opacity=0.75,
line=dict(width=0),
               colorbar=CBAR),
   hovertemplate="lon %{x:.3f}, lat %{y:.3f}<br>"
                 "RevPAN proxy
%{marker.color:,.1f}<extra></extra>"))
f4.update_layout(template=tpl(MODE), showlegend=False,
                 margin=dict(l=8, r=8, t=20, b=8),
                 xaxis=dict(title="longitude (grammar: bare
position channel)",
```

```
                           scaleanchor="y", scaleratio=1,
constrain="domain"),
                yaxis=dict(title="latitude", constrain="domain"))
save(f4, "fig4_grammar_scatter.png", w=MAP_W, h=MAP_H)


print("done ->", OUT)
```

- B.3 src/make_task2_figures.py

  Renders Figure 5.

```
"""
Report figure for Task 2 -- the aggregation funnel.


Shows the calendar table's row count collapsing across the three
semantic
layers (L0 atomic -> L1 aggregate -> L2 KPI), the central case
study for the
grain-misalignment argument in section 2.4.


Counts are read from the built parquet files rather than
hardcoded, so the
figure cannot silently drift away from the semantic layer it
describes.


Encoding note. This was a bar chart on a log x-axis. A bar asserts
that its
LENGTH is proportional to its value, and a log axis breaks exactly
that: the
53-row layer drew at ~40% the length of the 1.39M-row layer,
understating a
26,000x collapse by three orders of magnitude -- in the one figure
whose entire
job is to show the size of that collapse. Position on a log scale
is fine; a
length on one is not. Hence a lollipop: the marker's POSITION
carries the
magnitude, and the stem is a leader line to the label, not a
quantity.
```

```
Run:  .venv/bin/python src/make_task2_figures.py
"""

import math
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "app"))

import pandas as pd
import plotly.graph_objects as go

from theme import CATEGORICAL, CHROME, tpl

OUT = ROOT / "figures"
OUT.mkdir(exist_ok=True)
MODE = "light"
c, blue = CHROME[MODE], CATEGORICAL[MODE][0]

LAYERS = [
   ("L0 atomic<br><span style='font-size:11px'>one
listing-night</span>", "l0_calendar"),
   ("L1 aggregate<br><span style='font-size:11px'>listing x
month</span>", "l1_listing_month"),
   ("L2 KPI<br><span style='font-size:11px'>city x week</span>",
"l2_kpi_market"),
]
labels = [lab for lab, _ in LAYERS]
counts = [len(pd.read_parquet(ROOT / "data" / f"{t}.parquet")) for
_, t in LAYERS]

XMIN = 1
```

```
f = go.Figure()
# Stems: leader lines to the label, deliberately thin so they do
not read as bars.
for lab, v in zip(labels, counts):
    f.add_shape(type="line", x0=XMIN, x1=v, y0=lab, y1=lab,
                line=dict(color=c["grid"], width=2), layer="below")
f.add_trace(go.Scatter(
    x=counts, y=labels, mode="markers+text", orientation="h",
    marker=dict(size=15, color=blue, line=dict(width=2,
color=c["surface"])),
    text=[f"  {v:,}" for v in counts], textposition="middle right",
    textfont=dict(color=c["ink2"], size=12),
    hovertemplate="%{y}<br>%{x:,} rows<extra></extra>"))

# Annotate the two reductions -- the collapse is the message, so
state it in
# figures rather than leaving it to be eyeballed off a log axis.
# On a log axis, annotation x is given in log10 units.
for i in range(len(counts) - 1):
    f.add_annotation(
        x=(math.log10(counts[i]) + math.log10(counts[i + 1])) / 2,
        y=i + 0.5, xanchor="center", yanchor="middle",
showarrow=False,
        text=f"÷ {counts[i] / counts[i + 1]:,.0f}",
        font=dict(size=11.5, color=c["muted"]),
        bgcolor=c["surface"], borderpad=3)

f.update_layout(
    template=tpl(MODE), showlegend=False,
    margin=dict(l=8, r=8, t=20, b=8),
    xaxis=dict(type="log", title="Rows (log scale — position, not
length, carries the value)",
               showgrid=True, range=[0, 7.05]),
```

```
   yaxis=dict(showgrid=False, autorange="reversed",
ticklabelposition="outside"))

f.write_image(str(OUT / "fig5_grain_funnel.png"), width=820,
height=290, scale=2)

print("wrote fig5_grain_funnel.png", dict(zip([l.split("<br>")[0]
for l in labels], counts)))
```

- B.4 src/make_task3_figures.py

  Renders Figures 6-8.

```
"""
Report figures for Task 3 -- the two dashboards' interaction
idioms.


fig6/fig7 come from the EXPLORATORY dashboard (drill-down and
what-if);

fig8 comes from the EXPLANATORY one (act seven's flow diagram).
fig8 was

previously exported by hand and had no script, so it could not be
rebuilt from

the repo; it is generated here alongside the others.


Static exports lose the surrounding dashboard chrome, so the two
figures that

depend on filter state (fig7) carry that state in the figure
itself rather than

relying on a caption to supply it.


Run:  .venv/bin/python src/make_task3_figures.py

"""


import sys

from pathlib import Path


ROOT = Path(__file__).resolve().parent.parent

sys.path.insert(0, str(ROOT / "app"))
```

```
import exploratory as e

import explanatory as x

from theme import CHROME


OUT = ROOT / "figures"

OUT.mkdir(exist_ok=True)

MODE = "light"

c = CHROME[MODE]


W = len(e.WEEKS) - 1

base = dict(groups=None, rooms=None, price=[0,
e.OPTS["price_max"]], weeks=[0, W],

           sel={"neighbourhoods": []}, mode=MODE)


# ---- Fig 6: drill-down idiom (treemap, whole city)
------------------------

e._treemap(**base).write_image(str(OUT /
"fig6_exploratory_treemap.png"),

                               width=860, height=380, scale=2)

print("wrote fig6_exploratory_treemap.png")


# ---- Fig 7: what-if idiom (drilled to Belltown)
---------------------------

# On the dashboard the active selection is shown by a separate
chip and the

# y unit by the KPI tile beside it. Neither survives a PNG export,
so the

# figure was leaving a reader with an unlabelled y-axis and no way
to tell

# which neighbourhood it showed. Both are restated on the figure.

PICK = "Belltown"

drill = dict(base, sel={"neighbourhoods": [PICK]})

n_listings = len(e._slice(**{k: drill[k] for k in

                             ("groups", "rooms", "price", "weeks",
"sel")})[0])
```

```
f7 = e._whatif(**drill, dp=15, elast=1.5)
f7.update_layout(
   margin=dict(l=8, r=8, t=44, b=8),
   yaxis=dict(title="RevPAN proxy (index, not currency)",
rangemode="tozero"),
   xaxis=dict(showgrid=False, title=None),
   legend=dict(orientation="h", y=1.02, x=0.28))
f7.add_annotation(x=0, y=1.16, xref="paper", yref="paper",
xanchor="left",
                  showarrow=False, text=f"<b>{PICK}</b> ·
{n_listings} listings",
                  font=dict(size=12.5, color=c["ink"]))
f7.write_image(str(OUT / "fig7_exploratory_whatif.png"),
width=860, height=380, scale=2)
print("wrote fig7_exploratory_whatif.png (drilled to Belltown,
+15% @ e=1.5)")


# ---- Fig 8: explanatory flow idiom (Sankey, act seven)
--------------------
x._act7(MODE).write_image(str(OUT /
"fig8_explanatory_sankey.png"),
                          width=900, height=440, scale=2)
print("wrote fig8_explanatory_sankey.png")
```

- B.5 src/staticmap.py

  Server-side basemap tile fetch and compositing for Figure 3.

```
"""
Static raster basemap for report figures.


Why this exists
---------------
The dashboards draw their map with `px.scatter_map`, which renders
through
MapLibre on a WebGL canvas. That is correct in a browser, but
Kaleido's
```

```
headless Chrome exports the figure without a usable WebGL context: the map
canvas comes back empty, so the exported PNG has no basemap AND no markers --
only the SVG colourbar survives. That is what produced the blank
fig3_idiom_map.png.

The fix is to stop asking a headless browser to composite a map. Here the
basemap is assembled server-side instead: CARTO Positron raster tiles are
fetched over HTTP, placed as `layout.images` in Web-Mercator tile coordinates,
and the listings are drawn on top as an ordinary Cartesian scatter. Everything
then exports through Plotly's normal SVG/raster path, which Kaleido handles
reliably.

The result is the same idiom the dashboard shows -- same projection, same
sequential ramp, an equivalent light-grey canvas basemap -- but reproducible
offline once the tiles are cached in `.tilecache/`.

Tile source: Esri Light/Dark Gray Canvas rather than CARTO Positron. CARTO's
raster endpoint now stamps "API KEY REQUIRED" across every unkeyed tile, which
is unusable in a report; Esri's legacy ArcGIS Online canvas services are still
open and are the closest unkeyed match to Positron's styling.
"""

from __future__ import annotations
```

```
import math
from pathlib import Path

import requests

ROOT = Path(__file__).resolve().parent.parent
CACHE = ROOT / ".tilecache"

_ESRI =
("https://server.arcgisonline.com/ArcGIS/rest/services/Canvas/"
        "World_{v}_Gray_{layer}/MapServer/tile/{z}/{y}/{x}")

# Keyed by the app/theme.py map_style values so callers can pass
CHROME[mode]
# straight through. Each style is (base layer, label overlay).
TILE_URL = {
    "carto-positron": (_ESRI.replace("{v}",
"Light").replace("{layer}", "Base"),
                       _ESRI.replace("{v}",
"Light").replace("{layer}", "Reference")),
    "carto-darkmatter": (_ESRI.replace("{v}",
"Dark").replace("{layer}", "Base"),
                         _ESRI.replace("{v}",
"Dark").replace("{layer}", "Reference")),
}
ATTRIBUTION = "Basemap: Esri, HERE, Garmin  ·  © OpenStreetMap
contributors"



# ---------------------------------------------------------------
projection

def merc(lon: float, lat: float) -> tuple[float, float]:
    """Web-Mercator (EPSG:3857) normalised to the unit square, y
down."""
```

```
    x = lon / 360.0 + 0.5
    s = math.sin(math.radians(lat))
    y = 0.5 - math.log((1 + s) / (1 - s)) / (4 * math.pi)
    return x, y


def merc_x(lon):
    return lon / 360.0 + 0.5


def merc_y(lat):
    import numpy as np
    s = np.sin(np.radians(lat))
    return 0.5 - np.log((1 + s) / (1 - s)) / (4 * np.pi)


#
---------------------------------------------------------------------
tiles

def _tile(template: str, z: int, x: int, y: int) -> str | None:
    """Return a tile as a base64 data URI, fetching and caching on
first use.

    Returns None for a tile the service does not have (label
overlays are
    sparse -- most tiles carry no place name at all).
    """
    import base64
    import hashlib

    CACHE.mkdir(exist_ok=True)
    tag = hashlib.md5(template.encode()).hexdigest()[:8]
    stem = CACHE / f"{tag}_{z}_{x}_{y}"
```

```
   hit = next((p for p in (stem.with_suffix(".png"),
stem.with_suffix(".jpg"),
                           stem.with_suffix(".none")) if
p.exists()), None)
   if hit is None:
       r = requests.get(template.format(z=z, x=x, y=y),
timeout=30,
                        headers={"User-Agent":
"roy-assignment-figures/1.0"})
       ctype = r.headers.get("content-type", "")
       if r.status_code != 200 or "image" not in ctype:
           hit = stem.with_suffix(".none")
           hit.write_bytes(b"")
       else:
           hit = stem.with_suffix(".jpg" if "jpeg" in ctype else
".png")
           hit.write_bytes(r.content)
   if hit.suffix == ".none":
       return None
   mime = "image/jpeg" if hit.suffix == ".jpg" else "image/png"
   return f"data:{mime};base64," +
base64.b64encode(hit.read_bytes()).decode()



def _pick_zoom(span_x: float, device_px: float, max_zoom: int =
15) -> int:
   """Lowest zoom whose 256px tiles still out-resolve the output
raster."""
   for z in range(1, max_zoom + 1):
       if span_x * (2 ** z) * 256 >= device_px:
           return z
   return max_zoom
```

```
#
------------------------------------------------------------------
basemap


def basemap(fig, lon_range, lat_range, *, device_px,
style="carto-positron",
           attribution=True, labels=True):
   """Add tiled basemap images to `fig` and configure its
Cartesian axes.


   `lon_range` / `lat_range` are the geographic extents to cover.
The axes are
   left in Web-Mercator tile units (x east, y north) so that
`scaleanchor`
   with ratio 1 gives the correct, undistorted Mercator aspect.


   Returns a `project(lon, lat) -> (x, y)` callable in those same
tile units,
   which is what callers must use to place their own traces on
top.
   """
   x0, x1 = sorted(merc_x(v) for v in lon_range)
   y0, y1 = sorted(merc_y(v) for v in lat_range)      # y0 = north
edge


   z = _pick_zoom(x1 - x0, device_px)
   n = 2 ** z
   tx0, tx1 = int(x0 * n), int(x1 * n)
   ty0, ty1 = int(y0 * n), int(y1 * n)


   base_url, label_url = TILE_URL[style]
   # Base layer first, then labels on top -- Plotly draws layout
images in list
   # order. The label overlay is fetched one zoom level coarser
and stretched
```

```
    # over the 2x2 block it covers: place names are baked into the
raster at a
    # fixed pixel size, so at the base zoom they come out ~6px tall
and
    # illegible. Coarser tiles double them into something a reader
can use.
    layers = [(base_url, 0)] + ([(label_url, -1)] if labels else
[])

    # Tiles are placed edge to edge, which leaves a hairline seam
where two
    # anti-aliased image edges meet. A sub-pixel overlap closes it.
    eps = 0.004

    images = []
    for template, dz in layers:
        k = 2 ** -dz                        # tile side, in
base-zoom units
        for tx in range(tx0 // k, tx1 // k + 1):
            for ty in range(ty0 // k, ty1 // k + 1):
                src = _tile(template, z + dz, tx, ty)
                if src is None:
                    continue
                images.append(dict(
                    source=src, xref="x", yref="y",
                    x=tx * k - eps, y=-ty * k + eps,
                    sizex=k + 2 * eps, sizey=k + 2 * eps,
                    xanchor="left", yanchor="top",
                    sizing="stretch", layer="below", opacity=1))

    fig.update_layout(images=images)
    xr = [x0 * n, x1 * n]
    yr = [-y1 * n, -y0 * n]
```

```
    fig.update_xaxes(range=xr, showgrid=False, zeroline=False,
                     showticklabels=False, ticks="", title=None,
                     constrain="domain")
    fig.update_yaxes(range=yr, showgrid=False, zeroline=False,
                     showticklabels=False, ticks="", title=None,
                     scaleanchor="x", scaleratio=1,
constrain="domain")

    if attribution:
        fig.add_annotation(
            x=1, y=0, xref="paper", yref="paper", xanchor="right",
yanchor="bottom",
            text=ATTRIBUTION, showarrow=False,
            font=dict(size=9, color="#52514e"),
            bgcolor="rgba(255,255,255,0.72)", borderpad=3)

    def project(lon, lat):
        return merc_x(lon) * n, -merc_y(lat) * n

    return project



def fit_bounds(lon, lat, *, pad=0.06, aspect=None):
    """Data bounds padded by `pad`, optionally widened to a
plot-box aspect.

    `aspect` is plot-box height / width. Because the axes are
Mercator-square,
    matching the box aspect is what removes the dead margin that a
fixed
    `scaleanchor` otherwise leaves on the wide axis.
    """
    lo0, lo1 = float(min(lon)), float(max(lon))
    la0, la1 = float(min(lat)), float(max(lat))
```

```
    dlo, dla = (lo1 - lo0) * pad, (la1 - la0) * pad
    lo0, lo1, la0, la1 = lo0 - dlo, lo1 + dlo, la0 - dla, la1 + dla

    if aspect:
        x0, x1 = merc_x(lo0), merc_x(lo1)
        y0, y1 = float(merc_y(la1)), float(merc_y(la0))    # y0
north, y1 south
        w, h = x1 - x0, y1 - y0
        if h / w > aspect:                                 # too
tall -> widen x
            need = h / aspect
            cx = (x0 + x1) / 2
            lo0, lo1 = (cx - need / 2 - 0.5) * 360, (cx + need / 2
- 0.5) * 360
        else:                                              # too
wide -> heighten y
            need = w * aspect
            cy = (y0 + y1) / 2
            la1 = _inv_merc_y(cy - need / 2)
            la0 = _inv_merc_y(cy + need / 2)
    return (lo0, lo1), (la0, la1)



def _inv_merc_y(y: float) -> float:
    return math.degrees(2 * math.atan(math.exp((0.5 - y) * 2 *
math.pi)) - math.pi / 2)
```

- B.6 app/exploratory.py

  Dashboard 1 - exploratory, runs on port 8050.

```
"""
Dashboard 1 of 2 -- EXPLORATORY (user-driven).


Task 3 asks for a dashboard that "allows filtering, drill-down,
what-if
```

```
analysis". The controls are the point: nothing here argues a
thesis, it lets a
reader interrogate Seattle's short-let market and reach their own.

Coordination model
------------------
Every chart reads one shared filter state held in a dcc.Store.
Clicking a
treemap tile or a map point WRITES a neighbourhood into that
state, so
selection propagates to every other view -- the "coordinated
multiple views"
idiom. Colour is assigned by entity (room type), never by rank, so
narrowing
the filter never repaints the survivors.

Run:  .venv/bin/python app/exploratory.py   ->
http://127.0.0.1:8050
"""

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from dash import Dash, Input, Output, State, ctx, dcc, html,
no_update

import data_access as da
from styles import CSS
from theme import CATEGORICAL, CHROME, SEQUENTIAL, tpl
```

```
OPTS = da.options()
WEEKS = OPTS["weeks"]
ROOM_ORDER = OPTS["room_types"]                        # fixed order
-> fixed hues
# Hue follows the entity, not the mode's slot list: each room type
keeps its
# position in the fixed order, and the mode only decides which
step of that hue
# is drawn. Dark steps are selected for the dark surface, never an
auto-flip.
ROOM_COLOR = {mode: {r: CATEGORICAL[mode][i] for i, r in
enumerate(ROOM_ORDER)}
             for mode in ("light", "dark")}

SEQ = [[i / (len(SEQUENTIAL) - 1), h] for i, h in
enumerate(SEQUENTIAL)]



#
----------------------------------------------------------------------
helpers


def card(title, note, graph_id, height=330):
   return html.Div(className="card", children=[
       html.H2(title),
       html.P(note, className="note"),
       dcc.Graph(id=graph_id, config={"displayModeBar": False},
                 style={"height": f"{height}px"}),
   ])



def tile(key, value, caveat):
   return html.Div(className="tile", children=[
       html.Div(key, className="k"),
       html.Div(value, className="v"),
```

```
        html.Div(caveat, className="c"),
    ])



def blank(mode, msg):
    f = go.Figure()
    f.update_layout(template=tpl(mode), xaxis_visible=False,
yaxis_visible=False,
                    annotations=[dict(text=msg, showarrow=False,
font=dict(color=CHROME[mode]["muted"], size=13))])
    return f



#
-----------------------------------------------------------------
-- layout

app = Dash(__name__, title="Seattle Short-Let Explorer")
app.index_string = """<!DOCTYPE
html><html><head>{%metas%}<title>{%title%}</title>
{%favicon%}{%css%}<style>""" + CSS + """</style></head><body>
{%app_entry%}<footer>{%config%}{%scripts%}{%renderer%}</footer></b
ody></html>"""

app.layout = html.Div(className="wrap", children=[
    dcc.Store(id="sel", data={"neighbourhoods": []}),
    dcc.Store(id="mode", data="light"),

    html.Header([
        html.H1("Seattle short-let market — explorer"),
        html.Span("3,818 listings · 1.39M listing-nights · 84.8k
reviews", className="badge"),
        html.Button("Dark", id="toggle", className="toggle",
n_clicks=0),
```

```
    ]),
    html.P("User-driven view. Filter, then click any treemap tile
or map point to drill into a "
           "neighbourhood — every other chart follows. Metrics are
named for what they measure: "
           "the source prices only OPEN nights, so no revenue
figure is derivable and "
           "“blocked rate” is an upper bound on occupancy, not a
measurement of it.",
           className="sub"),

    # ---- filters: one row, above the charts ----
    html.Div(className="filters", children=[
        html.Div(className="f", children=[
            html.Label("Neighbourhood group"),
            dcc.Dropdown(id="f-group", options=OPTS["groups"],
multi=True,
                         placeholder="All 17 groups"),
        ]),
        html.Div(className="f", children=[
            html.Label("Room type"),
            dcc.Dropdown(id="f-room", options=ROOM_ORDER,
multi=True, placeholder="All types"),
        ]),
        html.Div(className="f", children=[
            html.Label("Listed price (USD/night)"),
            dcc.RangeSlider(id="f-price", min=0,
max=OPTS["price_max"], step=25,
                            value=[0, OPTS["price_max"]],
                            tooltip={"placement": "bottom",
"always_visible": False},
                            marks={0: "$0",
                                   int(OPTS["price_p99"]):
f"${int(OPTS['price_p99'])}",
                                   int(OPTS["price_max"]):
f"${int(OPTS['price_max'])}"}),
```

```
        ]),
        html.Div(className="f", children=[
            html.Label("Calendar window"),
            dcc.RangeSlider(id="f-week", min=0, max=len(WEEKS) - 1,
step=1,
                            value=[0, len(WEEKS) - 1],
                            marks={0:
str(pd.Timestamp(WEEKS[0]).date()),
                                   len(WEEKS) - 1:
str(pd.Timestamp(WEEKS[-1]).date())},
                            tooltip={"placement": "bottom",
"always_visible": False}),
        ]),
        html.Div(className="f", children=[
            html.Label("Drill-down"),
            html.Div(id="drill", className="hint"),
            html.Button("Clear selection", id="clear",
className="toggle",
                        style={"marginLeft": 0, "marginTop":
"6px"}, n_clicks=0),
        ]),
    ]),

    html.Div(id="tiles", className="tiles"),

    html.Div(className="grid g2", children=[
        card("Where the market sits", "Area sized by listing count,
shaded by RevPAN proxy. "
             "Click a tile to drill in — the rest of the board
follows.", "fig-treemap", 360),
        card("Where those listings are", "One point per listing,
shaded by RevPAN proxy. "
             "Click a point to select its neighbourhood.",
"fig-map", 360),
    ]),
```

```
    # Two measures of different scale -> two charts on one shared
x-axis.
    # Never a second y-axis: the alignment would be arbitrary and
would invent
    # a correlation the data does not contain.
    html.Div(className="grid g2", children=[
        card("Blocked rate, by week", "Share of nights unavailable.
Upper bound on occupancy — "
             "guest bookings and host blocks are indistinguishable
in this source.", "fig-blocked", 260),
        card("Asking price, by week", "Mean listed nightly price
across open nights. Plotted "
             "separately from blocked rate — a shared axis would
imply a link that isn't measured.",
             "fig-adr", 260),
    ]),

    html.Div(className="grid g2", children=[
        card("Room type across the city", "Three fixed hues
assigned by room type, so filtering "
             "never reassigns a colour. Exact values in the table
view below.", "fig-room", 320),
        card("What-if: repricing", "Constant-elasticity assumption,
not a fitted model — "
             "set the elasticity you want to defend and read the
response.", "fig-whatif", 320),
    ]),

    html.Div(className="filters", style={"gridTemplateColumns":
"1fr 1fr"}, children=[
        html.Div(className="f", children=[
            html.Label("Price change applied to every listing"),
            dcc.Slider(id="w-price", min=-30, max=30, step=5,
value=0,
                       marks={-30: "-30%", 0: "0", 30: "+30%"},
```

```
                    tooltip={"placement": "bottom",
"always_visible": True}),
        ]),
        html.Div(className="f", children=[
            html.Label("Assumed demand elasticity"),
            dcc.Slider(id="w-elast", min=0, max=2.5, step=0.25,
value=1.0,
                    marks={0: "0 (rigid)", 1: "1 (unit)", 2.5:
"2.5 (elastic)"},
                    tooltip={"placement": "bottom",
"always_visible": True}),
        ]),
    ]),

    html.Details([
        html.Summary("Table view — every filtered neighbourhood,
exact values"),
        html.Div(id="table"),
    ]),
])



# ------------------------------------------------------------------
interaction

@app.callback(Output("mode", "data"), Output("toggle",
"children"),
              Input("toggle", "n_clicks"))
def _toggle(n):
    return ("dark", "Light") if n % 2 else ("light", "Dark")



app.clientside_callback(
```

```
"function(m){document.documentElement.setAttribute('data-theme',
m); return '';}",
    Output("toggle", "title"), Input("mode", "data"))



@app.callback(Output("sel", "data"), Output("drill", "children"),
              Input("fig-treemap", "clickData"), Input("fig-map",
"clickData"),
              Input("clear", "n_clicks"), State("sel", "data"))
def _select(tree_click, map_click, _clear, sel):
    """Treemap and map both write the same selection key -- that
shared write is
    what makes the views coordinated rather than merely
adjacent."""
    trigger = ctx.triggered_id
    picked = list(sel.get("neighbourhoods", []))


    if trigger == "clear":
        picked = []
    elif trigger == "fig-treemap" and tree_click:
        label = tree_click["points"][0].get("label")
        parent = tree_click["points"][0].get("parent")
        if parent:                                      # a leaf, not
a group header
            picked = [] if label in picked else [label]
    elif trigger == "fig-map" and map_click:
        cd = map_click["points"][0].get("customdata")
        if cd:
            label = cd[0]
            picked = [] if label in picked else [label]


    msg = f"{picked[0]}" if picked else "Whole city — click to
drill in"
    return {"neighbourhoods": picked}, msg
```

```
FILTER_INPUTS = [Input("f-group", "value"), Input("f-room",
"value"),
                Input("f-price", "value"), Input("f-week",
"value"),
                Input("sel", "data"), Input("mode", "data")]



def _slice(groups, rooms, price, weeks, sel):
    """Resolve the shared filter state into the three frames the
charts need."""
    picked = (sel or {}).get("neighbourhoods") or None
    wk = (WEEKS[weeks[0]], WEEKS[weeks[1]])

    lst = da.apply_filters(da.listings(), groups, rooms, picked,
price)
    ids = set(lst["listing_id"])

    nw = da.neighbourhood_week()
    nw = da.apply_filters(nw, groups, None, picked, None, wk)

    lm = da.listing_month()
    lm = lm[lm["listing_id"].isin(ids)]
    return lst, nw, lm



@app.callback(Output("tiles", "children"), *FILTER_INPUTS)
def _tiles(groups, rooms, price, weeks, sel, mode):
    lst, nw, _ = _slice(groups, rooms, price, weeks, sel)
    if lst.empty or nw.empty:
        return [tile("No match", "—", "Widen the filters.")]
    agg = da.rollup(nw.assign(_all=1), "_all").iloc[0]
    return [
```

```
        tile("Listings", f"{len(lst):,}",
f"{lst['host_id'].nunique():,} distinct hosts"),
        tile("Blocked rate", f"{agg.blocked_rate:.1%}", "Upper
bound on occupancy"),
        tile("Asking ADR", f"${agg.asking_adr:,.0f}", "Open nights
only, not transacted"),
        tile("RevPAN proxy", f"{agg.revpan_proxy:,.1f}", "Index,
not currency"),
        tile("Sentiment", f"{lst['sentiment'].mean():.3f}", "VADER
compound, English only"),
    ]



@app.callback(Output("fig-treemap", "figure"), *FILTER_INPUTS)
def _treemap(groups, rooms, price, weeks, sel, mode):
    _, nw, _ = _slice(groups, rooms, price, weeks, sel)
    if nw.empty:
        return blank(mode, "No neighbourhoods match these filters")
    g = da.rollup(nw, ["neighbourhood_group", "neighbourhood"])
    g = g[g["listings"] > 0]
    f = px.treemap(g, path=[px.Constant("Seattle"),
"neighbourhood_group", "neighbourhood"],
                   values="listings", color="revpan_proxy",
                   color_continuous_scale=SEQ,
                   custom_data=["blocked_rate", "asking_adr",
"revpan_proxy"])
    f.update_traces(
        marker=dict(cornerradius=4, line=dict(width=2,
color=CHROME[mode]["surface"])),
        hovertemplate="<b>%{label}</b><br>%{value} listings<br>"
                      "Blocked %{customdata[0]:.1%}<br>Asking ADR
$%{customdata[1]:,.0f}<br>"
                      "RevPAN proxy
%{customdata[2]:,.1f}<extra></extra>",
        tiling=dict(pad=2))
    f.update_layout(
```

```
        template=tpl(mode), margin=dict(l=6, r=6, t=6, b=6),
        # 87 neighbourhoods means many tiles are too small for
their label.
        # Hide those rather than shrink them into illegibility --
the hover
        # and the table view carry the detail.
        uniformtext=dict(minsize=9, mode="hide"),
        coloraxis_colorbar=dict(title=dict(text="RevPAN<br>proxy",
font=dict(size=11)),
                                thickness=9, len=0.72,
outlinewidth=0,
                                tickfont=dict(size=10)))
    f.data[0].root = dict(color=CHROME[mode]["plane"])
    return f



@app.callback(Output("fig-map", "figure"), *FILTER_INPUTS)
def _map(groups, rooms, price, weeks, sel, mode):
    lst, _, lm = _slice(groups, rooms, price, weeks, sel)
    if lst.empty:
        return blank(mode, "No listings match these filters")
    d = lst.dropna(subset=["latitude", "longitude",
"revpan_proxy"])
    if d.empty:
        return blank(mode, "No priced listings in this selection")
    f = px.scatter_map(
        d, lat="latitude", lon="longitude", color="revpan_proxy",
        color_continuous_scale=SEQ, zoom=10.2, opacity=0.82,
        custom_data=["neighbourhood", "room_type", "price",
"blocked_rate", "revpan_proxy"])
    f.update_traces(
        marker=dict(size=8),
        hovertemplate="<b>%{customdata[0]}</b> ·
%{customdata[1]}<br>"
```

```
                      "Listed
$%{customdata[2]:,.0f}/night<br>Blocked %{customdata[3]:.1%}<br>"
                      "RevPAN proxy
%{customdata[4]:,.1f}<extra></extra>")
    f.update_layout(template=tpl(mode),
map_style=CHROME[mode]["map_style"],
                    margin=dict(l=0, r=0, t=0, b=0),

coloraxis_colorbar=dict(title=dict(text="RevPAN<br>proxy",
font=dict(size=11)),
                                            thickness=9, len=0.72,
outlinewidth=0,

tickfont=dict(size=10)))
    return f



def _weekly(mode, nw, col, fmt, label):
    if nw.empty:
        return blank(mode, "No calendar rows in this window")
    g = da.rollup(nw, "week").sort_values("week")
    c = CHROME[mode]
    f = go.Figure()
    f.add_trace(go.Scatter(
        x=g["week"], y=g[col], mode="lines",
        line=dict(width=2, color=CATEGORICAL[mode][0]), name=label,
        hovertemplate="%{x|%d %b %Y}<br>" + label + " " + fmt +
"<extra></extra>"))
    # Direct-label the endpoint only -- never a number on every
point.
    last = g.iloc[-1]
    f.add_trace(go.Scatter(
        x=[pd.Timestamp(last["week"]).to_pydatetime()],
y=[float(last[col])],
        mode="markers+text",
        marker=dict(size=8, color=CATEGORICAL[mode][0],
```

```
                  line=dict(width=2, color=c["surface"])),
        text=[f"{last[col]:.1%}" if col == "blocked_rate" else
f"${last[col]:,.0f}"],
        textposition="middle left", textfont=dict(color=c["ink2"],
size=11),
        showlegend=False, hoverinfo="skip"))
    f.update_layout(
        template=tpl(mode), showlegend=False, hovermode="x
unified",
        margin=dict(l=8, r=44, t=10, b=8),
        yaxis=dict(tickformat=".0%" if col == "blocked_rate" else
"$,.0f"),
        xaxis=dict(showgrid=False))
    return f



@app.callback(Output("fig-blocked", "figure"), *FILTER_INPUTS)
def _blocked(groups, rooms, price, weeks, sel, mode):
    _, nw, _ = _slice(groups, rooms, price, weeks, sel)
    return _weekly(mode, nw, "blocked_rate", "%{y:.1%}", "Blocked")



@app.callback(Output("fig-adr", "figure"), *FILTER_INPUTS)
def _adr(groups, rooms, price, weeks, sel, mode):
    _, nw, _ = _slice(groups, rooms, price, weeks, sel)
    return _weekly(mode, nw, "asking_adr", "$%{y:,.0f}", "Asking
ADR")



@app.callback(Output("fig-room", "figure"), *FILTER_INPUTS)
def _room(groups, rooms, price, weeks, sel, mode):
    lst, _, lm = _slice(groups, rooms, price, weeks, sel)
    if lm.empty:
        return blank(mode, "No listing-months match these filters")
```

```
    d = lm[lm["listing_id"].isin(set(lst["listing_id"]))]
    g = (d.groupby(["neighbourhood_group", "room_type"],
observed=True)
          .agg(nights=("nights", "sum"),
blocked=("blocked_nights", "sum"),
               psum=("price_sum", "sum"), open_n=("open_nights",
"sum"))
          .reset_index())
    g["blocked_rate"] = g["blocked"] / g["nights"]
    g["asking_adr"] = g["psum"] / g["open_n"].replace(0, pd.NA)
    g["revpan_proxy"] = g["asking_adr"] * g["blocked_rate"]
    g = g.dropna(subset=["revpan_proxy"])
    if g.empty:
        return blank(mode, "No priced nights in this selection")

    order =
(g.groupby("neighbourhood_group")["revpan_proxy"].mean()
             .sort_values(ascending=False).index.tolist())
    c = CHROME[mode]
    f = go.Figure()
    for rt in ROOM_ORDER:                              # fixed order
-> fixed hue
        s = g[g["room_type"] ==
rt].set_index("neighbourhood_group").reindex(order)
        if s["revpan_proxy"].isna().all():
            continue
        f.add_trace(go.Bar(
            x=order, y=s["revpan_proxy"], name=rt,
            marker=dict(color=ROOM_COLOR[mode][rt], cornerradius=4,
                        line=dict(width=2, color=c["surface"])),
            hovertemplate=f"<b>{rt}</b> · %{{x}}<br>RevPAN proxy
%{{y:,.1f}}<extra></extra>"))
    f.update_layout(template=tpl(mode), barmode="group",
bargap=0.28, bargroupgap=0.04,
                    margin=dict(l=8, r=8, t=8, b=8),
```

```
                  xaxis=dict(showgrid=False, tickangle=-32,
tickfont=dict(size=10)),
                  yaxis=dict(title=dict(text="RevPAN proxy")),
                  legend=dict(orientation="h", y=1.06, x=0))
   return f



@app.callback(Output("fig-whatif", "figure"), *FILTER_INPUTS,
              Input("w-price", "value"), Input("w-elast", "value"))
def _whatif(groups, rooms, price, weeks, sel, mode, dp, elast):
   """
   Constant-elasticity repricing. Demand responds as (1+dp)^-e, so
RevPAN
   scales by (1+dp)^(1-e): below unit elasticity a rise helps,
above it hurts,
   and at e=1 the two cancel exactly. Assumption, not a fitted
model -- the
   slider exists so the reader picks a number they can defend.
   """
   _, nw, _ = _slice(groups, rooms, price, weeks, sel)
   if nw.empty:
       return blank(mode, "No calendar rows in this window")
   g = da.rollup(nw, "week").sort_values("week")
   c, mult = CHROME[mode], 1 + dp / 100.0
   g["scenario"] = g["revpan_proxy"] * mult ** (1 - elast)
   g["scen_blocked"] = (g["blocked_rate"] * mult **
-elast).clip(upper=1.0)


   f = go.Figure()
   f.add_trace(go.Scatter(x=g["week"], y=g["revpan_proxy"],
mode="lines", name="Baseline",
                          line=dict(width=2,
color=CATEGORICAL[mode][0]),
                          hovertemplate="Baseline
%{y:,.1f}<extra></extra>"))
```

```
    f.add_trace(go.Scatter(x=g["week"], y=g["scenario"],
mode="lines",
                           name=f"{dp:+d}% at e={elast:g}",
                           line=dict(width=2,
color=CATEGORICAL[mode][1], dash="dot"),
                           hovertemplate="Scenario
%{y:,.1f}<extra></extra>"))
    delta = g["scenario"].mean() - g["revpan_proxy"].mean()
    f.add_annotation(x=1, y=1.16, xref="paper", yref="paper",
xanchor="right", showarrow=False,
                     text=f"Mean RevPAN proxy {delta:+,.1f}   ·
blocked rate "
                          f"{g['scen_blocked'].mean() -
g['blocked_rate'].mean():+.1%}",
                     font=dict(size=11.5, color=c["ink2"]))
    f.update_layout(template=tpl(mode), hovermode="x unified",
                    margin=dict(l=8, r=8, t=30, b=8),
xaxis=dict(showgrid=False),
                    legend=dict(orientation="h", y=1.02, x=0))
    return f



@app.callback(Output("table", "children"), *FILTER_INPUTS)
def _table(groups, rooms, price, weeks, sel, mode):
    """Relief for the sub-3:1 contrast slot: exact values, never
colour alone."""
    _, nw, _ = _slice(groups, rooms, price, weeks, sel)
    if nw.empty:
        return html.P("Nothing matches these filters.",
className="note")
    g = (da.rollup(nw, ["neighbourhood_group", "neighbourhood"])
           .sort_values("revpan_proxy", ascending=False).head(40))
    head = ["Neighbourhood", "Group", "Listings", "Blocked rate",
"Asking ADR", "RevPAN proxy"]
    rows = [html.Tr([
```

```
        html.Td(r.neighbourhood), html.Td(r.neighbourhood_group,
style={"textAlign": "left"}),
        html.Td(f"{int(r.listings):,}"),
html.Td(f"{r.blocked_rate:.1%}"),
        html.Td("—" if pd.isna(r.asking_adr) else
f"${r.asking_adr:,.0f}"),
        html.Td("—" if pd.isna(r.revpan_proxy) else
f"{r.revpan_proxy:,.1f}"),
    ]) for r in g.itertuples()]
    return html.Table([html.Thead(html.Tr([html.Th(h) for h in
head])), html.Tbody(rows)])



if __name__ == "__main__":
    app.run(debug=False, port=8050)
```

- B.7 app/[explanatory.py]

  Dashboard 2 - explanatory, runs on port 8051.

```
"""
Dashboard 2 of 2 -- EXPLANATORY (story-driven).


Same semantic layer as the explorer, opposite posture. There are
no filters:
the sequence is fixed because the argument is fixed. Each act
states a claim,
shows the figure that tests it, and names what the figure does not
license.


The argument
------------
The most striking pattern in this dataset -- blocked rate falling
from 52% to
27% across 2016 -- is an ARTIFACT of how the data was collected,
not a market
trend. Acts 1-2 set up the trap and spring it; acts 3-5 establish
what the data
```

```
does support; act 6 draws the decision-making consequence.


Run:  .venv/bin/python app/explanatory.py   ->
http://127.0.0.1:8051
"""


import sys
from pathlib import Path


sys.path.insert(0, str(Path(__file__).resolve().parent))


import pandas as pd
import plotly.graph_objects as go
from dash import Dash, Input, Output, dcc, html


import data_access as da
import story_data as sd
from styles import CSS, STORY_CSS
from theme import CATEGORICAL, CHROME, FONT, STATUS, tpl


SPEARMAN = sd.horizon_spearman()
SENT = sd.sentiment_spread()


# Fixed room-type order and per-mode hue -- mirrors
app/exploratory.py so a
# room type keeps the same colour identity across both dashboards.
ROOM_ORDER = da.options()["room_types"]
ROOM_COLOR = {mode: {r: CATEGORICAL[mode][i] for i, r in
enumerate(ROOM_ORDER)}
              for mode in ("light", "dark")}
```

```
# ---------------------------------------------------------------------
building


def act(n, kicker, title, paras, finding, fig_id, caption,
height=340):
    body = [html.Div(kicker, className="act-n"), html.H2(title)]
    body += [html.P(p) for p in paras]
    if finding:
        body.append(html.Div(finding, className="finding"))
    body += [
        html.Div(className="figure", children=[
            dcc.Graph(id=fig_id, config={"displayModeBar": False},
                      style={"height": f"{height}px"})]),
        html.P(caption, className="caption"),
        html.Hr(className="rule"),
    ]
    return html.Section(className="act", children=body)



app = Dash(__name__, title="What Seattle's Calendar Does Not Say")
app.index_string = """<!DOCTYPE
html><html><head>{%metas%}<title>{%title%}</title>
{%favicon%}{%css%}<style>""" + CSS + STORY_CSS +
"""</style></head><body>
{%app_entry%}<footer>{%config%}{%scripts%}{%renderer%}</footer></b
ody></html>"""


app.layout = html.Div(className="wrap", children=[
    dcc.Store(id="mode", data="light"),

    html.Div(className="story", children=[
        html.Header([
            html.H1("What Seattle's calendar does not say"),
```

```
            html.Button("Dark", id="toggle", className="toggle",
n_clicks=0),
        ]),
        html.P("A guided read of 1.39 million listing-nights, in
six steps. The headline "
               "number in this dataset is real, reproducible, and
means almost nothing — "
               "and the way it fools you is a property of the
visualisation grammar, not "
               "of the analyst.", className="lede"),
        html.Hr(classNam="rule"),

        act(1, "Act one · the headline",
            "Seattle's short-let supply appears to loosen all
year",
            ["Plot the share of nights marked unavailable, week by
week, exactly as the "
             "source presents it. The line falls from 52% in early
January to 27% by the "
             "end of December — a 25-point slide, with a clean
summer interruption.",
             "Read conventionally, this is a straightforward market
story: demand softening "
             "through the year, hosts opening more inventory, a
July peak. Every convention "
             "of the time-series idiom encourages that reading.
Time on x, rate on y, one "
             "continuous line: the form itself asserts that the
change is temporal."],
            "Nothing about this chart is wrong. The data is
accurate and the encoding is "
            "standard. The conclusion it invites is still false.",
            "fig-act1", "Weekly share of nights flagged
unavailable, all 3,818 listings. "
            "Source: calendar.csv aggregated to ISO weeks.", 300),

        act(2, "Act two · the reveal",
```

```
           "The x-axis is not time. It is distance from a single
scrape.",
           ["calendar.csv is not a year of observations. It is one
forward-looking snapshot "
            "taken on 4 January 2016, listing what each host had
open for the following "
            "365 days. Nights close to that date had months to
accumulate bookings and "
            "blocks. Nights in December 2016 had almost none — not
because demand was "
            "weak, but because nobody had booked them yet.",
            "Replot the identical data against
days-ahead-of-scrape and the shape resolves "
            f"into a booking curve — noisy and stepped, but
unmistakably downward. Spearman's ρ "
            "between horizon and blocked rate is "
            f"{SPEARMAN:.2f}: the further out you look, the
emptier the calendar, by "
            "construction."],
           "The decline is a measurement horizon, drawn on a time
axis. Anyone reading Act "
           "One as a demand signal is reading the collection
method.",
           "fig-act2", "Identical rows to Act One, re-encoded
against booking horizon. "
           "Faint marks are daily values; the line is a 14-day
centred mean.", 320),


       act(3, "Act three · the control",
           "Reviews are written after the stay, so they cannot
carry the artifact",
           ["To find real seasonality you need a series the scrape
cannot contaminate. "
            "Reviews qualify: a guest writes one after departing,
so review dates record "
            "stays that already happened. Three full years
(2013–2015, 60,000 reviews) sit "
```

```
            "entirely before the scrape.",
            "The seasonal signal is enormous and unambiguous.
August runs 1.85× the annual "
            "mean; February runs 0.37×. That is a five-fold swing
between the strongest and "
            "weakest months — far larger than anything the
calendar chart suggested."],
           "Seattle's short-let market is powerfully seasonal. The
calendar chart, read "
           "naively, understated that swing and misattributed its
direction.",
           "fig-act3", "Monthly review counts 2013–2015, indexed
to the annual mean. "
           "Backward-looking and independent of the 2016 scrape.",
300),


       act(4, "Act four · what survives",
           "Summer demand is strong enough to push through the
artifact",
           ["Now test the two against each other. If the booking
curve were the only thing "
            "in the calendar, blocked rate would decay
monotonically with horizon. It does "
            "not. The May–August window sits at 33–38%, above BOTH
the stretch before it "
            "(29% at 60–90 days) and the stretch after it (28% at
300–365). A bump that "
            "clears its neighbours on both sides is not a decay
artifact.",
            "It is also corroborated: the review series, built
from entirely different rows, "
            "independently puts July and August at 1.49× and 1.85×
the annual mean. Two "
            "unrelated measurements agreeing on the same peak is
much harder to dismiss "
            "than either alone.",
            "One honest caveat. The summer level arrives as a step
on 1 July (+6.8 points "
```

```
            "in a day), not as a gradual build — consistent with
hosts and guests treating "
            "the season as a block rather than accumulating
bookings smoothly. A second, "
            "larger step on 1 February (+9.5 points) has no such
explanation and remains "
            "unaccounted for."],
           "Summer elevation is real and doubly evidenced. Its
shape — a step, not a ramp — "
           "is a finding in its own right, and one anomaly is left
unexplained rather than "
           "smoothed away.",
           "fig-act4", "Blocked rate by horizon with the
May–August window marked. Shaded "
           "band spans days 120–240 ahead of the scrape. The
visible steps at 1 Apr, "
           "1 Jul are day-level discontinuities, not smoothing
artifacts.", 320),


       act(5, "Act five · the metric itself",
           "A booking-driven market peaks on Saturday. This one is
flat.",
           ["One more check, and it undercuts the metric rather
than the trend. Short-let "
            "bookings concentrate hard on Friday and Saturday
nights. If the unavailable "
            "flag mostly meant 'a guest booked this', the
day-of-week profile would show "
            "it plainly.",
            "It does not. Every weekday sits between 32.7% and
33.2% — a spread of half a "
            "percentage point across the entire week. Multi-day
host blocks, which span "
            "whole weeks indiscriminately, dominate the signal."],
           "'Unavailable' does not mean 'booked'. It is an upper
bound on occupancy that "
```

```
            "is mostly measuring something else, and no amount of
chart polish fixes that.",
            "fig-act5", "Share of nights flagged unavailable by
weekday, all 1.39M "
            "listing-nights. Axis starts at zero.", 290),

        act(6, "Act six · the consequence",
            "Where price and turnover actually diverge",
            ["Strip out the artifact and one durable geographic
pattern remains. Ranking "
             "each neighbourhood by asking price and, separately,
by review velocity — a "
             "proxy for how fast it turns over — the two rankings
come apart at both ends.",
             "Pike-Market and Alki price in the top decile and turn
over in the bottom "
             "quintile. Mid-Beacon Hill, Crown Hill and Columbia
City do the reverse. "
             "Across all 46 neighbourhoods the correlation is weak
(r = -0.11), so this is "
             "not a law — it is a shortlist of places where the
pricing and the demand "
             "disagree enough to be worth a look."],
            "This is the one finding here that survives every
check, and it came from the "
            "backward-looking series — not from the 1.39 million
rows that looked more "
            "authoritative.",
            "fig-act6", "Percentile rank of asking price versus
review velocity, "
            "neighbourhoods with ≥20 listings. Ordered by the gap
between them.", 430),

        act(7, "Act seven · the shape of the market",
            "Where the inventory actually sits, end to end",
            ["One last idiom, better suited to composition than any
chart so far: a flow "
```

```
            "diagram from geography through room type to price
tier. Every one of the "
            "3,818 listings passes through exactly once — the five
largest neighbourhood "
            "groups keep their name, the rest fold into one node
so the diagram stays "
            "readable rather than technically complete.",
            "The shape says what no single-panel chart could: the
premium tier is fed "
            "almost exclusively by entire-home listings — 97% of
every $200+ listing is "
            "an entire home, versus 3% private room and
effectively zero shared. Read the "
            "other way, private- and shared-room listings
themselves land in the budget "
            "tier 85% and 96% of the time respectively;
entire-home listings split three "
            "ways (28% budget, 52% mid, 20% premium) and are the
only room type that "
            "reaches premium at meaningful volume at all.
Geography barely changes that "
            "split; it is a property of room type, not of
neighbourhood."],
           "Premium pricing power belongs almost entirely to
entire-home listings. A "
           "pricing or inventory strategy organised by
neighbourhood alone is optimising "
           "the wrong axis for the tier that carries the most
revenue potential per night.",
           "fig-act7", "Listing count flowing from neighbourhood
group, through room "
           "type, to asking-price tier. Link colour follows room
type — the entity the "
           "flow is really about — not the neighbourhood it
happens to pass through.", 420),


       html.Div(className="closing", children=[
           html.Div("So what", className="kicker"),
```

```
           html.H2("Four things this dataset will let you say"),
           html.Ol([
               html.Li("Seattle short-lets swing roughly five-fold
between February and "
                       "August. Plan staffing, pricing and
inventory against that, not "
                       "against the calendar's apparent annual
drift."),
               html.Li("Peak-season nights commit six to eight
months ahead. A pricing "
                       "decision made in March has already missed
part of the summer."),
               html.Li("Do not report occupancy from this source.
The unavailable flag is "
                       "dominated by host blocks; the honest
ceiling is an upper bound, and "
                       "the flat weekday profile proves it."),
               html.Li("Treat Pike-Market, Alki and South Lake
Union as priced ahead of "
                       "their turnover, and the Beacon Hill
corridor as the reverse — as "
                       "hypotheses to test, not conclusions."),
           ]),
           html.P("And one it will not: that supply loosened
across 2016. That chart is "
                  "reproducible, defensible, and an artifact of a
single January scrape.",
                  style={"marginTop": "16px"}),
       ]),
   ]),
])



#
------------------------------------------------------------------
the acts
```

```
@app.callback(Output("mode", "data"), Output("toggle",
"children"),
              Input("toggle", "n_clicks"))
def _toggle(n):
    return ("dark", "Light") if n % 2 else ("light", "Dark")



app.clientside_callback(

"function(m){document.documentElement.setAttribute('data-theme',
m); return '';}",
    Output("toggle", "title"), Input("mode", "data"))



def _base(fig, mode, **kw):
    fig.update_layout(template=tpl(mode),
showlegend=kw.pop("legend", False),
                      margin=kw.pop("margin", dict(l=8, r=8, t=34,
b=8)), **kw)
    return fig



@app.callback(Output("fig-act1", "figure"), Input("mode", "data"))
def _act1(mode):
    g = da.kpi_market().sort_values("week")
    c, blue = CHROME[mode], CATEGORICAL[mode][0]
    f = go.Figure()
    f.add_trace(go.Scatter(x=g["week"], y=g["blocked_rate"],
mode="lines",
                           line=dict(width=2, color=blue),
                           hovertemplate="%{x|%d %b %Y}<br>Blocked
%{y:.1%}<extra></extra>"))
    for i, lbl in ((0, "52%"), (len(g) - 1, "27%")):
        r = g.iloc[i]
```

```
        f.add_trace(go.Scatter(
            x=[pd.Timestamp(r["week"]).to_pydatetime()],
y=[float(r["blocked_rate"])],
            mode="markers+text", text=[lbl],
            textposition="bottom right" if i == 0 else "top left",
            textfont=dict(color=c["ink2"], size=12),
            marker=dict(size=9, color=blue, line=dict(width=2,
color=c["surface"])),
            hoverinfo="skip"))
    f.add_annotation(x=0.52, y=0.9, xref="paper", yref="paper",
showarrow=False,
                     text="reads as a year-long decline",
font=dict(size=12, color=c["muted"]))
    return _base(f, mode, hovermode="x unified",
                 yaxis=dict(tickformat=".0%", rangemode="tozero"),
                 xaxis=dict(showgrid=False))



@app.callback(Output("fig-act2", "figure"), Input("mode", "data"))
def _act2(mode):
    g = sd.horizon_curve()
    c, blue = CHROME[mode], CATEGORICAL[mode][0]
    f = go.Figure()
    f.add_trace(go.Scatter(x=g["horizon"], y=g["blocked_rate"],
mode="markers",
                           marker=dict(size=3.5, color=blue,
opacity=0.28),
                           hoverinfo="skip"))
    f.add_trace(go.Scatter(x=g["horizon"], y=g["blocked_smooth"],
mode="lines",
                           line=dict(width=2, color=blue),
                           hovertemplate="%{x} days
ahead<br>Blocked %{y:.1%}<extra></extra>"))
    f.add_annotation(x=300, y=g["blocked_smooth"].iloc[-40],
ax=-70, ay=-46,
```

```
                    text=f"ρ = {SPEARMAN:.2f}<br>further out → emptier",
                    font=dict(size=12, color=c["ink2"]), align="left",
                    arrowcolor=c["axis"], arrowwidth=1, arrowsize=1, arrowhead=0)
    return _base(f, mode, hovermode="x unified",
                 yaxis=dict(tickformat=".0%", rangemode="tozero"),
                 xaxis=dict(title=dict(text="Days ahead of the 4 Jan 2016 scrape"),
                            showgrid=False))


@app.callback(Output("fig-act3", "figure"), Input("mode", "data"))
def _act3(mode):
    g = sd.review_seasonality()
    c, blue = CHROME[mode], CATEGORICAL[mode][0]
    f = go.Figure()
    f.add_trace(go.Bar(
        x=g["name"], y=g["index"],
        marker=dict(color=blue, cornerradius=4, line=dict(width=2, color=c["surface"])),
        text=[f"{v:.2f}×" if n in ("Aug", "Feb") else "" for v, n in zip(g["index"], g["name"])],
        textposition="outside", textfont=dict(color=c["ink2"], size=12),
        hovertemplate="<b>%{x}</b><br>%{y:.2f}× the annual mean<extra></extra>"))
    f.add_hline(y=1.0, line=dict(width=1, color=c["axis"]),
                annotation_text="annual mean", annotation_position="right",
                annotation_font=dict(size=11, color=c["muted"]))
    return _base(f, mode, bargap=0.32,
                 yaxis=dict(title=dict(text="Index (1.0 = mean)"), rangemode="tozero"),
                 xaxis=dict(showgrid=False))
```

```
@app.callback(Output("fig-act4", "figure"), Input("mode", "data"))
def _act4(mode):
    g = sd.horizon_curve()
    c, blue = CHROME[mode], CATEGORICAL[mode][0]
    f = go.Figure()
    f.add_vrect(x0=120, x1=240, fillcolor=blue, opacity=0.07,
line_width=0,
                annotation_text="May – Aug",
annotation_position="top left",
                annotation_font=dict(size=11.5, color=c["muted"]))
    f.add_trace(go.Scatter(x=g["horizon"], y=g["blocked_smooth"],
mode="lines",
                           line=dict(width=2, color=blue),
                           hovertemplate="%{x} days
ahead<br>Blocked %{y:.1%}<extra></extra>"))
    f.add_annotation(x=185, y=float(g.loc[g.horizon.between(170,
200), "blocked_smooth"].max()),
                     ax=54, ay=-42, text="bends up against<br>its
own trend",
                     font=dict(size=12, color=c["ink2"]),
align="left",
                     arrowcolor=c["axis"], arrowwidth=1,
arrowsize=1, arrowhead=0)
    return _base(f, mode, hovermode="x unified",
                 yaxis=dict(tickformat=".0%", rangemode="tozero"),
                 xaxis=dict(title=dict(text="Days ahead of the 4
Jan 2016 scrape"),
                            showgrid=False))


@app.callback(Output("fig-act5", "figure"), Input("mode", "data"))
def _act5(mode):
    g = sd.dow_curve()
```

```
    c = CHROME[mode]
    # Status colour, not a series colour: the point of this chart is that a
    # metric is not measuring what its name claims.
    f = go.Figure()
    f.add_trace(go.Bar(
        x=g["name"], y=g["is_booked"],
        marker=dict(color=STATUS["warning"], cornerradius=4,
                    line=dict(width=2, color=c["surface"])),
        hovertemplate="<b>%{x}</b><br>Blocked %{y:.2%}<extra></extra>"))
    f.add_annotation(x=0.5, y=0.86, xref="paper", yref="paper", showarrow=False,
                     text="⚠ 0.5-point spread across the whole week — "
                          "no weekend peak to be found",
                     font=dict(size=12, color=c["ink2"]))
    return _base(f, mode, bargap=0.34,
                 yaxis=dict(tickformat=".0%", rangemode="tozero", range=[0, 0.5],
                            title=dict(text="Share of nights unavailable")),
                 xaxis=dict(showgrid=False))


@app.callback(Output("fig-act6", "figure"), Input("mode", "data"))
def _act6(mode):
    g = sd.price_turnover_gap()
    g = pd.concat([g.head(6), g.tail(6)]).sort_values("gap")
    c = CHROME[mode]
    blue, orange = CATEGORICAL[mode][0], CATEGORICAL[mode][1]
    f = go.Figure()
    for r in g.itertuples():                          # connector first, dots on top
        f.add_trace(go.Scatter(
```

```
            x=[r.adr_pct, r.vel_pct], y=[r.neighbourhood,
r.neighbourhood],
            mode="lines", line=dict(width=1.5, color=c["grid"]),
            showlegend=False, hoverinfo="skip"))
    f.add_trace(go.Scatter(
        x=g["adr_pct"], y=g["neighbourhood"], mode="markers",
name="Asking price rank",
        marker=dict(size=11, color=blue, line=dict(width=2,
color=c["surface"])),
        hovertemplate="<b>%{y}</b><br>Price rank
%{x:.0%}<extra></extra>"))
    f.add_trace(go.Scatter(
        x=g["vel_pct"], y=g["neighbourhood"], mode="markers",
name="Turnover rank",
        marker=dict(size=11, symbol="diamond", color=orange,
                    line=dict(width=2, color=c["surface"])),
        hovertemplate="<b>%{y}</b><br>Turnover rank
%{x:.0%}<extra></extra>"))
    return _base(f, mode, legend=True,
                 margin=dict(l=8, r=8, t=44, b=8),
                 xaxis=dict(tickformat=".0%", range=[-0.04, 1.04],
                            title=dict(text="Percentile rank among
46 neighbourhoods")),
                 yaxis=dict(showgrid=False,
tickfont=dict(size=11.5)))



def _rgba(hex_color, alpha):
    h = hex_color.lstrip("#")
    r, g, b = (int(h[i:i + 2], 16) for i in (0, 2, 4))
    return f"rgba({r},{g},{b},{alpha})"



PRICE_TIERS = ["Budget (<$100)", "Mid ($100–199)", "Premium
($200+)"]
```

```
def _price_tier(p):
    if p < 100:
        return PRICE_TIERS[0]
    if p < 200:
        return PRICE_TIERS[1]
    return PRICE_TIERS[2]



@app.callback(Output("fig-act7", "figure"), Input("mode", "data"))
def _act7(mode):
    """
    Three-stage flow: neighbourhood group -> room type -> price
tier.

    Groups fold to the top 5 by listing count plus one "Rest of
Seattle" node --
    the same fold-past-N-categories rule used everywhere else in
this project,
    applied here so a 17-group source stays readable as a diagram
rather than
    merely complete. Link colour follows room type, the middle
stage, because
    that is the entity whose identity the diagram is actually
tracing.
    """
    lst = da.listings().dropna(subset=["neighbourhood_group",
"room_type", "price"]).copy()
    # The source already ships a literal "Other neighborhoods"
bucket -- exclude
    # it from the ranking so it folds into "Rest of Seattle" rather
than sitting
    # beside it as a second, confusingly similar catch-all node.
    named = lst.loc[lst["neighbourhood_group"] != "Other
neighborhoods", "neighbourhood_group"]
```

```
    top_groups = named.value_counts().head(5).index.tolist()
    lst["group"] = lst["neighbourhood_group"].where(
        lst["neighbourhood_group"].isin(top_groups), "Rest of
Seattle")
    lst["tier"] = lst["price"].map(_price_tier)

    groups =
lst.groupby("group").size().sort_values(ascending=False).index.tol
ist()
    rooms = ROOM_ORDER
    tiers = PRICE_TIERS
    nodes = groups + rooms + tiers
    idx = {n: i for i, n in enumerate(nodes)}

    c = CHROME[mode]
    room_hex = ROOM_COLOR[mode]
    node_color = ([c["axis"]] * len(groups) + [room_hex[r] for r in
rooms]
                  + [c["axis"]] * len(tiers))

    stage1 = lst.groupby(["group",
"room_type"]).size().reset_index(name="n")
    stage2 = lst.groupby(["room_type",
"tier"]).size().reset_index(name="n")

    src, tgt, val, link_color = [], [], [], []
    for r in stage1.itertuples():
        src.append(idx[r.group]); tgt.append(idx[r.room_type]);
val.append(r.n)
        link_color.append(_rgba(room_hex[r.room_type], 0.28))
    for r in stage2.itertuples():
        src.append(idx[r.room_type]); tgt.append(idx[r.tier]);
val.append(r.n)
        link_color.append(_rgba(room_hex[r.room_type], 0.28))
```

```
    f = go.Figure(go.Sankey(
        arrangement="snap",
        node=dict(label=nodes, color=node_color, pad=14,
thickness=14,
                  line=dict(width=0),
                  hovertemplate="<b>%{label}</b><br>%{value:,}
listings<extra></extra>"),
        link=dict(source=src, target=tgt, value=val,
color=link_color,
                  hovertemplate="<b>%{source.label} ->
%{target.label}</b><br>"
                                "%{value:,}
listings<extra></extra>"),
        textfont=dict(color=c["ink"], size=12, family=FONT),
    ))
    f.update_layout(template=tpl(mode), margin=dict(l=8, r=8, t=8,
b=8),
                    font=dict(color=c["ink"]))
    return f



if __name__ == "__main__":
    app.run(debug=False, port=8051)
```

- B.8 app/data_access.py

  Cached Parquet loaders shared by both dashboards.

```
"""
Read-side of the semantic layer.


Frames are loaded once at import and treated as immutable; every
filter returns
a copy. Dashboard callbacks never touch the raw CSVs -- they read
the Parquet
tables built by src/build_semantic_layer.py, which is what keeps
interaction
responsive on 1.4M atomic rows.
```

```
Metric names are deliberate. See build_semantic_layer.build_11 for
why there is
no revenue column and why occupancy is called `blocked_rate`.
"""

from functools import lru_cache
from pathlib import Path

import pandas as pd

DATA = Path(__file__).resolve().parent.parent / "data"

# Human-facing metric labels + the caveat each one carries.
METRICS = {
   "blocked_rate": ("Blocked rate", "Share of nights unavailable.
Conflates guest-booked "
                                    "with host-blocked, so it is
an upper bound on occupancy."),
   "asking_adr": ("Asking ADR", "Mean listed nightly price across
OPEN nights only. "
                                "Not a transacted rate -- blocked
nights carry no price."),
   "revpan_proxy": ("RevPAN proxy", "Asking ADR x blocked rate. A
comparative index for "
                                    "ranking listings and areas,
not a currency amount."),
   "review_velocity": ("Review velocity", "Reviews per listing per
month. A demand proxy: "
                                          "only a minority of
stays leave a review."),
   "sentiment_index": ("Sentiment", "Mean VADER compound score,
English reviews only. "
                                    "Heavily left-skewed -- read
deviations, not levels."),
}
```

```python
@lru_cache(maxsize=None)
def load(name: str) -> pd.DataFrame:
    return pd.read_parquet(DATA / f"{name}.parquet")


def listings() -> pd.DataFrame:
    return load("l2_listing_scorecard")


def listing_month() -> pd.DataFrame:
    return load("l1_listing_month")


def neighbourhood_week() -> pd.DataFrame:
    return load("l1_neighbourhood_week")


def review_month() -> pd.DataFrame:
    return load("l1_review_month")


def kpi_neighbourhood() -> pd.DataFrame:
    return load("l2_kpi_neighbourhood")


def kpi_market() -> pd.DataFrame:
    return load("l2_kpi_market")


def amenity_penetration() -> pd.DataFrame:
    return load("l2_amenity_penetration")
```

```
def options():
    """Static filter domains, computed once."""
    lst = listings()
    nw = neighbourhood_week()
    return {
        "groups":
sorted(lst["neighbourhood_group"].dropna().unique().tolist()),
        "room_types":
sorted(lst["room_type"].dropna().unique().tolist()),
        "weeks": sorted(nw["week"].unique().tolist()),
        "price_max": float(lst["price"].max()),
        "price_p99": float(lst["price"].quantile(0.99)),
    }



def apply_filters(df, groups=None, room_types=None,
neighbourhoods=None,
                  price_range=None, week_range=None):
    """
    Shared predicate pushdown. Each clause is skipped when the
caller passes
    nothing, so the same function serves every chart regardless of
which
    columns that chart's frame happens to carry.
    """
    out = df
    if groups and "neighbourhood_group" in out.columns:
        out = out[out["neighbourhood_group"].isin(groups)]
    if room_types and "room_type" in out.columns:
        out = out[out["room_type"].isin(room_types)]
    if neighbourhoods and "neighbourhood" in out.columns:
        out = out[out["neighbourhood"].isin(neighbourhoods)]
```

```
    if price_range and "price" in out.columns:
        lo, hi = price_range
        out = out[out["price"].between(lo, hi)]
    if week_range and "week" in out.columns:
        lo, hi = week_range
        out = out[out["week"].between(pd.Timestamp(lo),
pd.Timestamp(hi))]
    return out



def rollup(nw: pd.DataFrame, by) -> pd.DataFrame:
    """
    Re-derive metrics from atomic sums at whatever grain `by`
names.

    Aggregating the ratio columns directly would break the identity
    revpan_proxy == asking_adr * blocked_rate wherever price and
availability
    correlate, so ratios are always recomputed from the underlying
counts.
    """
    g = (nw.groupby(by, observed=True)
           .agg(listings=("listings", "max"),
                nights=("nights", "sum"),
                blocked_nights=("blocked_nights", "sum"),
                price_sum=("price_sum", "sum"),
                open_nights=("open_nights", "sum"))
           .reset_index())
    g["blocked_rate"] = g["blocked_nights"] / g["nights"]
    g["asking_adr"] = g["price_sum"] / g["open_nights"].replace(0,
pd.NA)
    g["revpan_proxy"] = g["asking_adr"] * g["blocked_rate"]
    return g
```

- B.9 app/story_data.py

Pre-computed narrative figures for the explanatory dashboard.

```
"""
Derived frames for the explanatory dashboard.

The story turns on one fact about the source: calendar.csv is a
SINGLE forward
scrape taken on 2016-01-04, not a year of observations. Every
frame here is
built to test, rather than assume, what that implies.
"""

from functools import lru_cache

import numpy as np
import pandas as pd

import data_access as da

SCRAPE = pd.Timestamp("2016-01-04")



@lru_cache(maxsize=None)
def horizon_curve() -> pd.DataFrame:
    """Blocked rate and asking price by days-ahead-of-scrape.

    If a forward scrape drives the weekly trend, blocked rate must
decay with
    horizon: near nights have had time to accumulate bookings and
blocks, far
    nights have not. This frame is what makes that testable.
    """
    c = da.load("10_calendar")
    h = c.assign(horizon=(c["date"] - SCRAPE).dt.days)
    g = (h.groupby("horizon")
```

```
          .agg(blocked_rate=("is_booked", "mean"),
               asking_adr=("price", "mean"),
               n=("is_booked", "size"))
          .reset_index())
   g["blocked_smooth"] = g["blocked_rate"].rolling(14,
center=True, min_periods=1).mean()
   g["adr_smooth"] = g["asking_adr"].rolling(14, center=True,
min_periods=1).mean()
   g["month"] = (SCRAPE + pd.to_timedelta(g["horizon"],
unit="D")).dt.month
   return g



@lru_cache(maxsize=None)
def horizon_spearman() -> float:
   g = horizon_curve()
   return float(g[["horizon",
"blocked_rate"]].corr(method="spearman").iloc[0, 1])



@lru_cache(maxsize=None)
def review_seasonality() -> pd.DataFrame:
   """Monthly review index from 2013-2015.

   Reviews are written AFTER a stay, so they are backward-looking
and carry no
   forward-scrape artifact at all. That independence is the whole
point: it is
   the control against which the calendar's apparent trend is
judged.
   """
   r = da.load("l0_review")
   r = r[r["date"].dt.year.between(2013, 2015)]
   s = r.groupby(r["date"].dt.month).size()
   return pd.DataFrame({
```

```
        "month": s.index,
        "index": (s / s.mean()).values,
        "reviews": s.values,
        "name": [pd.Timestamp(2016, m, 1).strftime("%b") for m in
s.index],
    })


@lru_cache(maxsize=None)
def dow_curve() -> pd.DataFrame:
    """Blocked rate by day of week.

    A market driven by guest bookings peaks hard on Fri/Sat. A flat
profile is
    evidence the 'f' flag is dominated by multi-day host blocks
instead.
    """
    c = da.load("l0_calendar")
    g = c.groupby("dow")["is_booked"].mean().reset_index()
    g["name"] = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
    return g


@lru_cache(maxsize=None)
def price_turnover_gap(min_listings: int = 20) -> pd.DataFrame:
    """Rank divergence between what a neighbourhood asks and how
fast it turns over.

    Levels are not comparable across a 2.5x price range, so both
measures are
    converted to percentile ranks and the story is the gap between
them.
    """
    k = da.kpi_neighbourhood()
```

```
    k = k[k["listings"] >= min_listings].copy()
    k["adr_pct"] = k["asking_adr"].rank(pct=True)
    k["vel_pct"] = k["review_velocity"].rank(pct=True)
    k["gap"] = k["adr_pct"] - k["vel_pct"]
    return k.sort_values("gap")



@lru_cache(maxsize=None)
def sentiment_spread() -> dict:
    k = da.kpi_neighbourhood()
    s = k["sentiment_index"].dropna()
    return {"min": float(s.min()), "max": float(s.max()),
            "sd": float(s.std()), "mean": float(s.mean())}
```

- B.10 app/theme.py

  Shared Plotly template, colourway, and scale definitions.

```
"""
Palette, Plotly templates and shared chart chrome.


Values come from the validated reference palette. The categorical
order is
fixed (blue, orange, aqua) and assigned by entity, never by rank,
so filtering
never repaints the survivors. Only the first three slots are used:
they are the
set that clears the all-pairs CVD and normal-vision floors in both
modes, which
is what map and scatter forms require.


Validator run (light, --pairs all, surface #fcfcfb):
    lightness band PASS | chroma floor PASS
    CVD separation PASS  worst aqua<->orange dE 9.2 (deutan)
    normal-vision  PASS  worst aqua<->blue   dE 24.0
    contrast       WARN  aqua 2.74:1 -> relief rule: every
categorical series
```

```
                        carries a visible direct label AND a table
view.
"""


import plotly.graph_objects as go
import plotly.io as pio


# --- categorical: fixed order, assigned by entity
-------------------------
CATEGORICAL = {
   "light": ["#2a78d6", "#eb6834", "#1baf7a"],
   "dark": ["#3987e5", "#d95926", "#199e70"],
}


# --- sequential: one hue, light -> dark (magnitude only)
-------------------
SEQUENTIAL = ["#cde2fb", "#9ec5f4", "#6da7ec", "#3987e5",
"#256abf", "#184f95", "#0d366b"]


# --- diverging: warm/cool poles, neutral gray midpoint
--------------------
DIVERGING = {
   "light": [[0.0, "#0d366b"], [0.5, "#f0efec"], [1.0,
"#d03b3b"]],
   "dark": [[0.0, "#3987e5"], [0.5, "#383835"], [1.0, "#e66767"]],
}


STATUS = {"good": "#0ca30c", "warning": "#fab219", "serious":
"#ec835a", "critical": "#d03b3b"}


CHROME = {
   "light": {
       "surface": "#fcfcfb", "plane": "#f9f9f7",
       "ink": "#0b0b0b", "ink2": "#52514e", "muted": "#898781",
       "grid": "#e1e0d9", "axis": "#c3c2b7",
```

```
        "border": "rgba(11,11,11,0.10)",
        "map_style": "carto-positron",
    },
    "dark": {
        "surface": "#1a1a19", "plane": "#0d0d0d",
        "ink": "#ffffff", "ink2": "#c3c2b7", "muted": "#898781",
        "grid": "#2c2c2a", "axis": "#383835",
        "border": "rgba(255,255,255,0.10)",
        "map_style": "carto-darkmatter",
    },
}

FONT = 'system-ui, -apple-system, "Segoe UI", sans-serif'



def _template(mode: str) -> go.layout.Template:
    c = CHROME[mode]
    axis = dict(
        showgrid=True, gridcolor=c["grid"], gridwidth=1,
griddash="solid",
        zeroline=False, showline=True, linecolor=c["axis"],
linewidth=1,
        ticks="outside", ticklen=4, tickcolor=c["axis"],
        tickfont=dict(color=c["muted"], size=11),
        title=dict(font=dict(color=c["ink2"], size=12)),
        automargin=True,
    )
    return go.layout.Template(layout=dict(
        colorway=CATEGORICAL[mode],
        paper_bgcolor=c["surface"], plot_bgcolor=c["surface"],
        font=dict(family=FONT, color=c["ink"], size=12),
        title=dict(font=dict(size=14, color=c["ink"]), x=0,
xanchor="left", y=0.97),
        xaxis=axis, yaxis=axis,
```

```
        margin=dict(l=8, r=8, t=44, b=8),
        hoverlabel=dict(
            bgcolor=c["surface"], bordercolor=c["axis"],
font=dict(family=FONT, size=12, color=c["ink"])
        ),
        legend=dict(
            orientation="h", yanchor="bottom", y=1.02,
xanchor="left", x=0,
            font=dict(size=11, color=c["ink2"]),
bgcolor="rgba(0,0,0,0)",
            title=dict(text=""),
        ),
        colorscale=dict(sequential=[[i / (len(SEQUENTIAL) - 1), h]
for i, h in enumerate(SEQUENTIAL)],
                        diverging=DIVERGING[mode]),
    ))



pio.templates["viz_light"] = _template("light")
pio.templates["viz_dark"] = _template("dark")



def tpl(mode: str) -> str:
    return f"viz_{mode}"
```

- B.11 app/styles.py

  Shared CSS-in-Python style constants for the Dash layouts.

```
"""Page chrome CSS. Both themes are declared explicitly; dark is a
selected
set of steps from the same ramps, not an inverted light."""


from theme import CHROME, FONT


def _vars(mode):
    c = CHROME[mode]
```

```
    return "".join(f"--{k.replace('_','-')}:{v};" for k, v in
c.items() if k != "map_style")


CSS = f"""
:root {{ color-scheme: light; {_vars('light')} }}
:root[data-theme="dark"] {{ color-scheme: dark; {_vars('dark')} }}
@media (prefers-color-scheme: dark) {{
 :root:not([data-theme="light"]) {{ color-scheme: dark;
{_vars('dark')} }}
}}
* {{ box-sizing: border-box; }}
body {{
 margin: 0; background: var(--plane); color: var(--ink);
 font-family: {FONT}; font-size: 14px; -webkit-font-smoothing:
antialiased;
}}
.wrap {{ max-width: 1500px; margin: 0 auto; padding: 20px 22px
56px; }}
header {{ display: flex; align-items: baseline; gap: 14px;
flex-wrap: wrap; margin-bottom: 4px; }}
h1 {{ font-size: 19px; font-weight: 650; margin: 0;
letter-spacing: -0.01em; }}
.sub {{ color: var(--ink2); font-size: 13px; margin: 0 0 18px;
max-width: 78ch; line-height: 1.5; }}
.badge {{
 font-size: 11px; color: var(--ink2); border: 1px solid
var(--border);
 border-radius: 999px; padding: 2px 9px; background:
var(--surface);
}}


/* --- filters: one row above the charts --- */
.filters {{
 display: grid; grid-template-columns: repeat(auto-fit,
minmax(215px, 1fr));
 gap: 14px; align-items: end;
```

```
 background: var(--surface); border: 1px solid var(--border);
 border-radius: 10px; padding: 14px 16px; margin-bottom: 16px;
}}
.f label {{
 display: block; font-size: 11px; font-weight: 600;
letter-spacing: .04em;
 text-transform: uppercase; color: var(--muted); margin-bottom:
6px;
}}
.hint {{ font-size: 11px; color: var(--muted); margin-top: 5px; }}

/* --- stat tiles: the number IS the chart --- */
.tiles {{ display: grid; grid-template-columns: repeat(auto-fit,
minmax(168px, 1fr)); gap: 12px; margin-bottom: 16px; }}
.tile {{
 background: var(--surface); border: 1px solid var(--border);
 border-radius: 10px; padding: 13px 15px;
}}
.tile .k {{ font-size: 11px; font-weight: 600; letter-spacing:
.04em; text-transform: uppercase; color: var(--muted); }}
.tile .v {{ font-size: 27px; font-weight: 640; margin-top: 5px;
letter-spacing: -0.02em; line-height: 1.1; }}
.tile .c {{ font-size: 11px; color: var(--ink2); margin-top: 6px;
line-height: 1.4; }}

/* --- chart cards --- */
.grid {{ display: grid; gap: 14px; margin-bottom: 14px; }}
.g2 {{ grid-template-columns: 1fr 1fr; }}
.card {{
 background: var(--surface); border: 1px solid var(--border);
 border-radius: 10px; padding: 6px 8px 4px; min-width: 0;
}}
.card h2 {{ font-size: 13px; font-weight: 620; margin: 10px 10px
2px; }}
```

```
.card p.note {{ font-size: 11.5px; color: var(--ink2); margin: 4px
10px 8px; line-height: 1.45; }}

@media (max-width: 1000px) {{ .g2 {{ grid-template-columns: 1fr;
}} }}


/* --- table view: the relief for the sub-3:1 contrast slot --- */

table {{ width: 100%; border-collapse: collapse; font-size:
12.5px; font-variant-numeric: tabular-nums; }}

th, td {{ text-align: right; padding: 7px 10px; border-bottom: 1px
solid var(--grid); }}

th:first-child, td:first-child {{ text-align: left;
font-variant-numeric: normal; }}

th {{ color: var(--muted); font-weight: 600; font-size: 11px;
text-transform: uppercase; letter-spacing: .04em; }}

.swatch {{ display: inline-block; width: 9px; height: 9px;
border-radius: 2px; margin-right: 7px; vertical-align: baseline;
}}

details {{ background: var(--surface); border: 1px solid
var(--border); border-radius: 10px; padding: 10px 14px; }}

summary {{ cursor: pointer; font-size: 12.5px; font-weight: 600;
color: var(--ink2); }}

details[open] summary {{ margin-bottom: 10px; }}


/* --- theme toggle --- */

.toggle {{

 margin-left: auto; background: var(--surface); color:
var(--ink2);

 border: 1px solid var(--border); border-radius: 8px;

 padding: 6px 13px; font-size: 12px; cursor: pointer; font-family:
inherit;

}}

.toggle:hover {{ color: var(--ink); }}


/* Dash controls inherit the surface rather than the library
default white */

.Select-control, .Select-menu-outer, .rc-slider-tooltip-inner {{
```

```
 background: var(--surface) !important; color: var(--ink)
!important;
 border-color: var(--border) !important;
}}
.Select-value-label, .Select-placeholder {{ color: var(--ink)
!important; }}
.Select--multi .Select-value {{
 background: var(--plane) !important; border-color: var(--border)
!important; color: var(--ink) !important;
}}
"""



STORY_CSS = """
.story { max-width: 940px; margin: 0 auto; }
.act { margin: 0 0 60px; }
.act-n {
 font-size: 11px; font-weight: 700; letter-spacing: .12em;
text-transform: uppercase;
 color: var(--muted); margin-bottom: 8px;
}
.act h2 { font-size: 21px; font-weight: 640; margin: 0 0 10px;
letter-spacing: -0.015em; line-height: 1.25; }
.act p { font-size: 14.5px; line-height: 1.62; color: var(--ink2);
margin: 0 0 14px; max-width: 68ch; }
.act p strong { color: var(--ink); font-weight: 620; }
.finding {
 border-left: 3px solid var(--accent); padding: 3px 0 3px 16px;
 margin: 18px 0; font-size: 15.5px; line-height: 1.55; color:
var(--ink); font-weight: 500;
}
.figure {
 background: var(--surface); border: 1px solid var(--border);
 border-radius: 10px; padding: 8px 8px 4px; margin: 18px 0 10px;
}
```

```
.caption { font-size: 11.5px; color: var(--muted); line-height:
1.5; margin: 2px 4px 12px; }

.rule { border: 0; border-top: 1px solid var(--grid); margin: 0 0
52px; }

.lede {

 font-size: 17px; line-height: 1.6; color: var(--ink2); max-width:
70ch; margin: 0 0 8px;

}

.kicker {

 display: inline-block; font-size: 11px; font-weight: 700;
letter-spacing: .12em;

 text-transform: uppercase; color: var(--accent); margin-bottom:
10px;

}

.closing {

 background: var(--surface); border: 1px solid var(--border);
border-radius: 10px;

 padding: 22px 26px; margin-top: 8px;

}

.closing h2 { margin-top: 0; }

.closing ol { margin: 0; padding-left: 20px; }

.closing li { font-size: 14.5px; line-height: 1.6; color:
var(--ink2); margin-bottom: 10px; max-width: 66ch; }

:root { --accent: #2a78d6; }

:root[data-theme="dark"] { --accent: #3987e5; }

@media (prefers-color-scheme: dark) {

 :root:not([data-theme="light"]) { --accent: #3987e5; }

}

"""
```